Master : Farhad Hosseinzade Lotfi

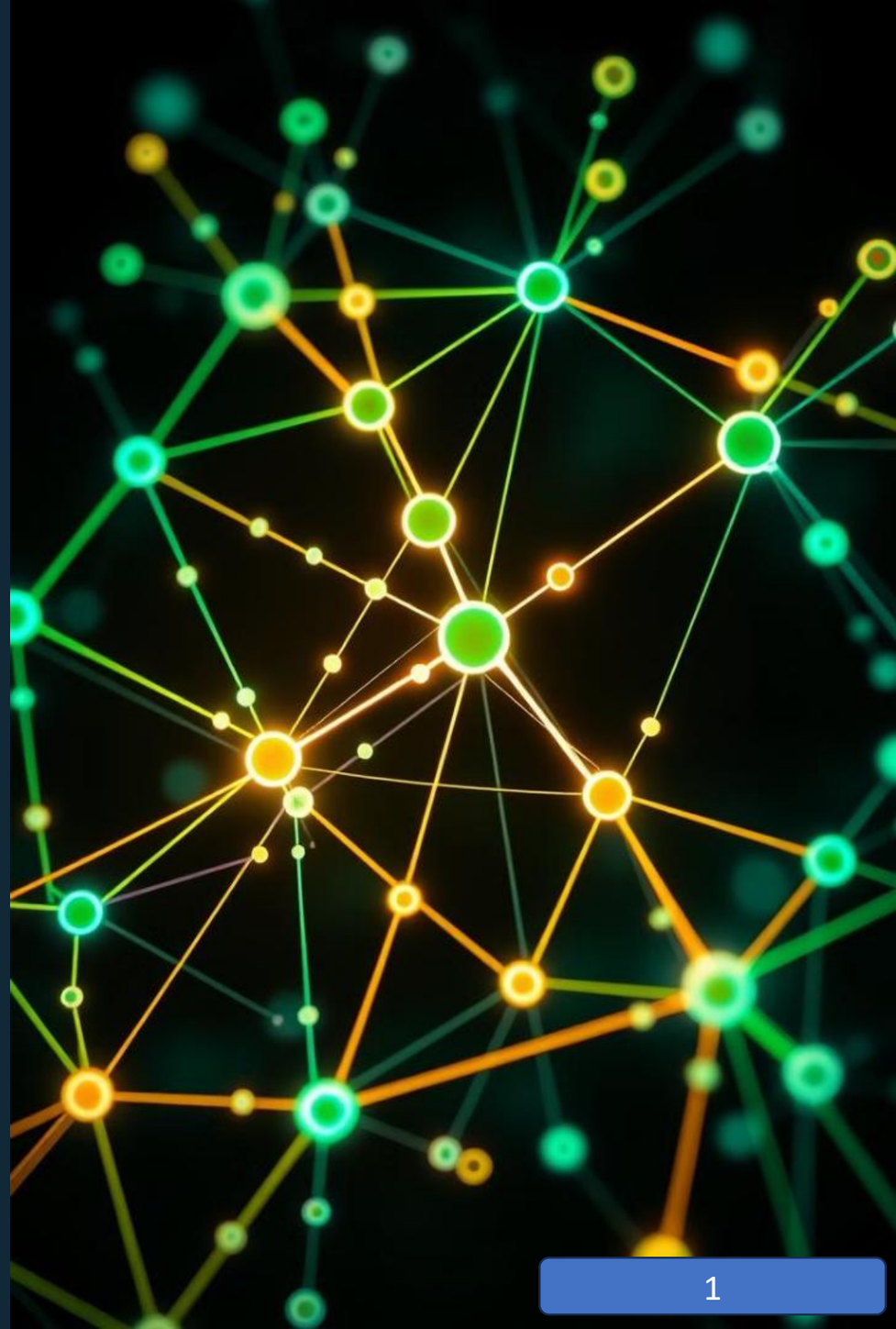
Student : Ali Nanaee

Project

Finding the best path in a graph

Understanding Graphs: A Comprehensive Tutorial

This presentation delves into the fundamental concepts of graphs, exploring their definitions, types, representations, and applications in computer science.



Graph Terminology

Nodes

Represent entities or objects. Often called vertices.

Edges

Represent connections or relationships between nodes.

Directed Edges

Edges with a specific direction, indicating a one-way relationship.

Undirected Edges

Edges without direction, indicating a two-way relationship.

Types of Graphs

Directed

Edges have direction, indicating relationships with an order.

Undirected

Edges have no direction, indicating relationships without order.

Weighted

Edges have associated values, representing costs, distances, or weights.

Unweighted

Edges have no associated values, indicating simple connections.

Representing Graphs

Adjacency Matrix

A 2D array where rows and columns represent nodes. Each cell indicates the presence or absence of an edge.

Adjacency List

A list where each element corresponds to a node. Each element contains a list of adjacent nodes.

Edge List

A list of edges, where each edge is represented as a pair of nodes.

Graph Data Structures

1 Nodes

Can store various attributes like data, labels, or properties.

2 Edges

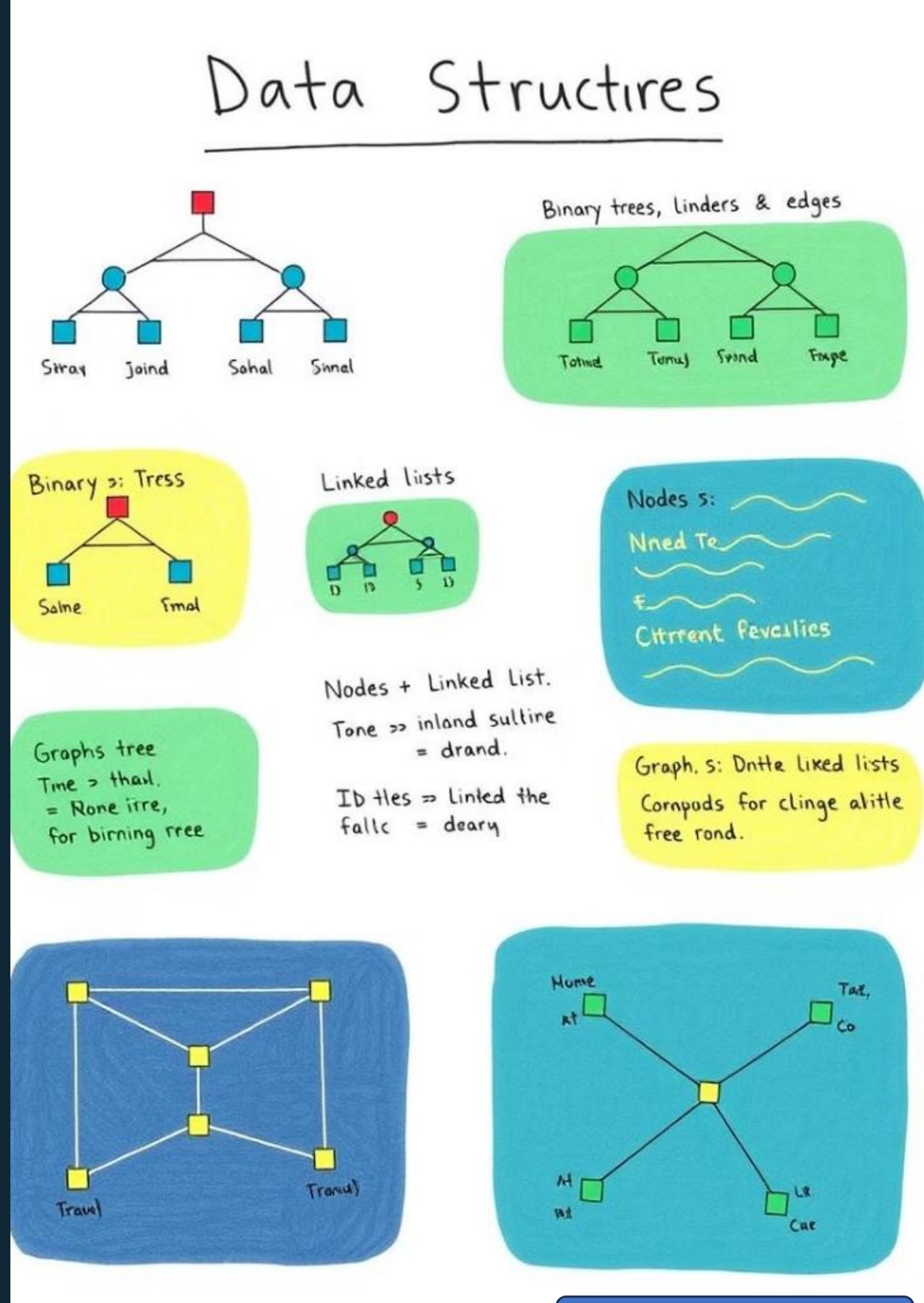
Can also store attributes like weights, costs, or capacities.

3 Adjacency Matrix

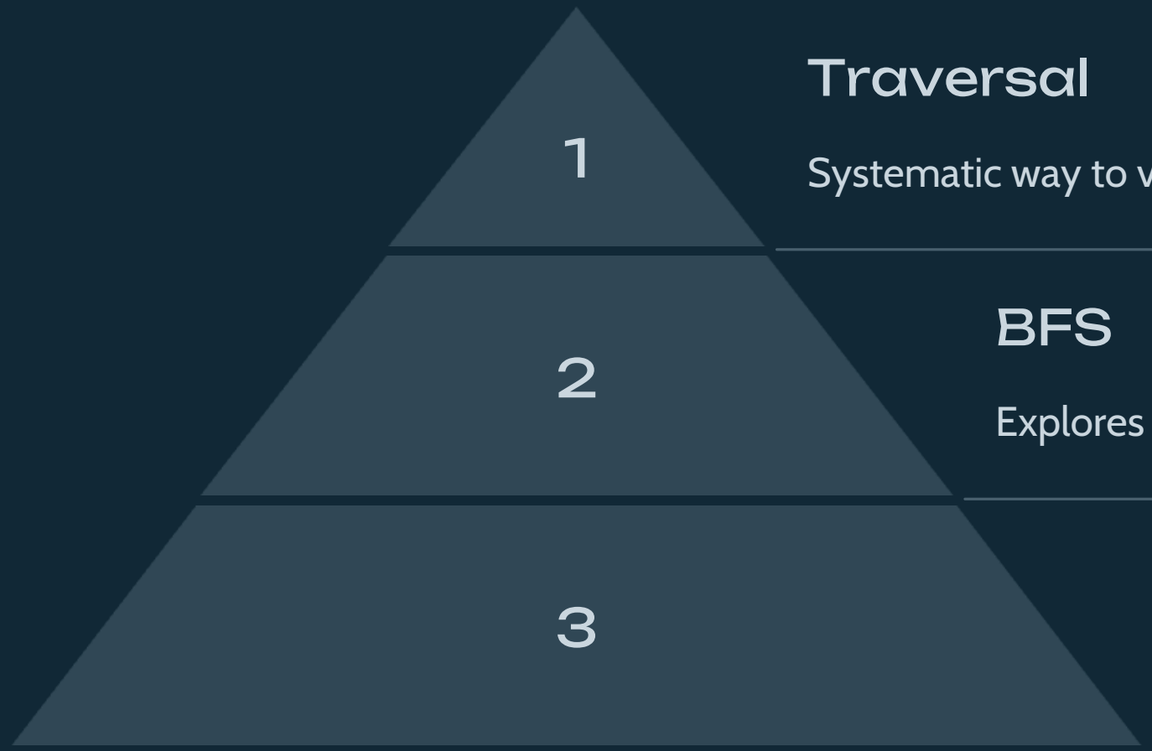
Represents connections with binary values, indicating presence or absence of edges.

4 Adjacency List

Stores connections as lists of adjacent nodes, efficient for sparse graphs.



Graph Traversal Algorithms



Traversal

Systematic way to visit all nodes in a graph.

BFS

Explores nodes level by level.

DFS

Explores nodes in depth first order.

Breadth-First Search (BFS)

1

Start Node

Choose the starting node for the search.

2

Queue

Store unvisited nodes in a queue.

3

Visit Neighbors

Visit the neighbors of the current node.

4

Repeat

Repeat the process until all reachable nodes are visited.

Depth-First Search (DFS)

Stack

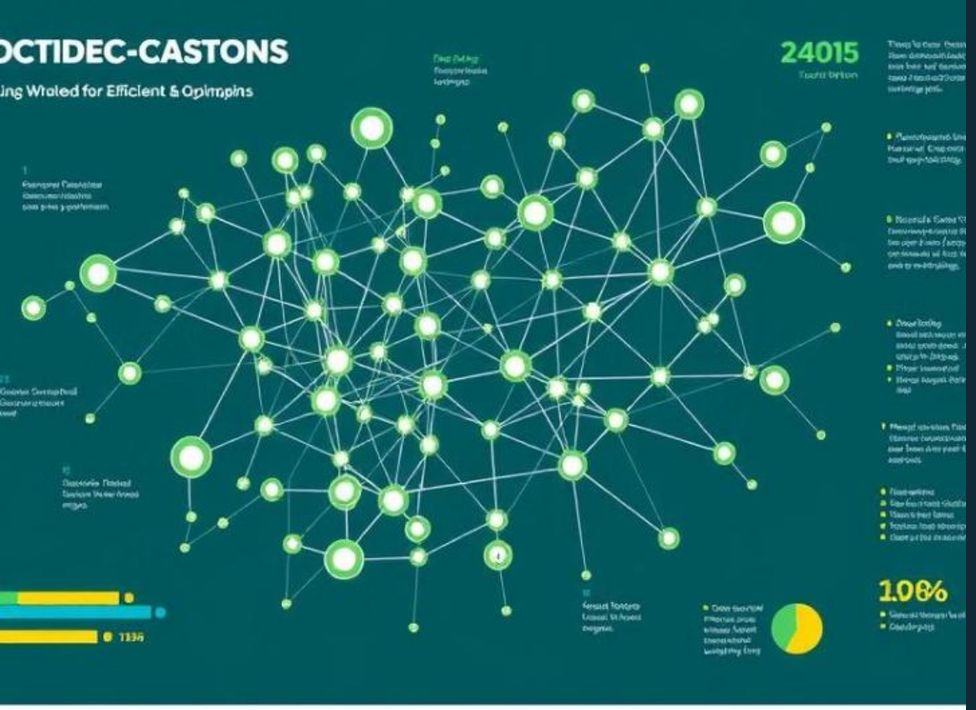
Store unvisited nodes in a stack.

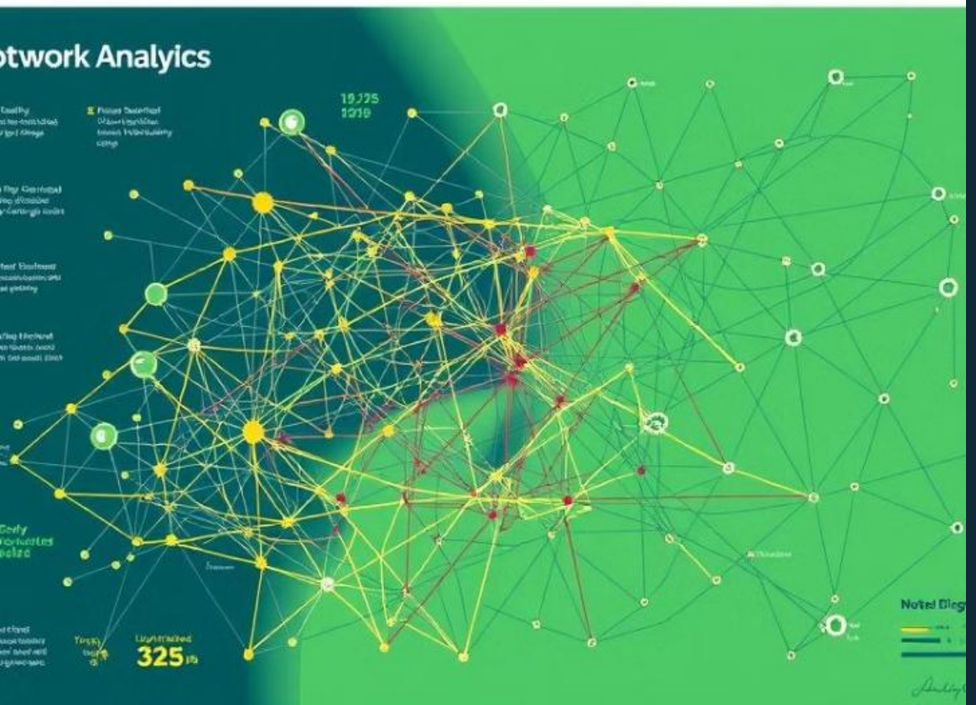
Recursive Call

Recursively visit neighbors of the current node.

Backtrack

If no unvisited neighbors, backtrack to the previous node.





Graph Applications

1

Social Networks

Modeling connections between people.

2

Transportation

Route planning and shortest path finding.

3

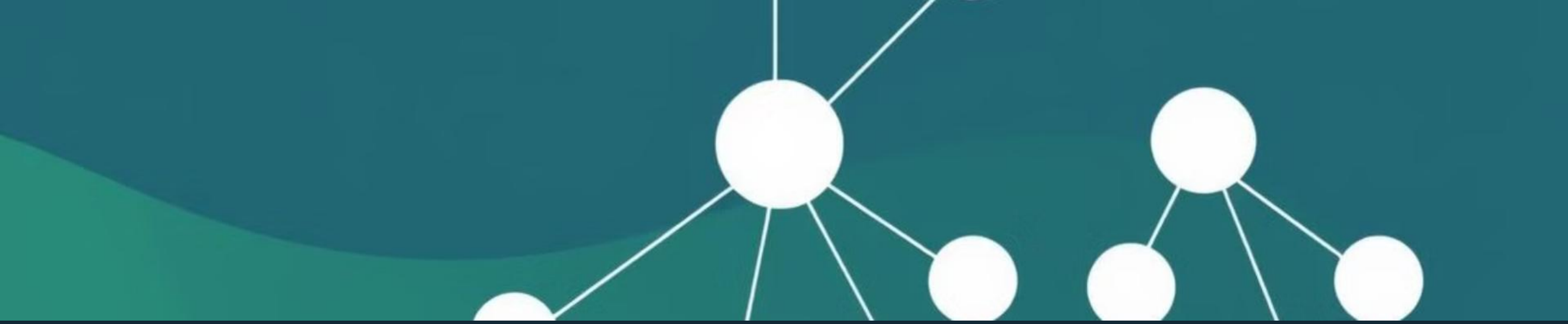
Network Analysis

Analyzing network structure and connectivity.

4

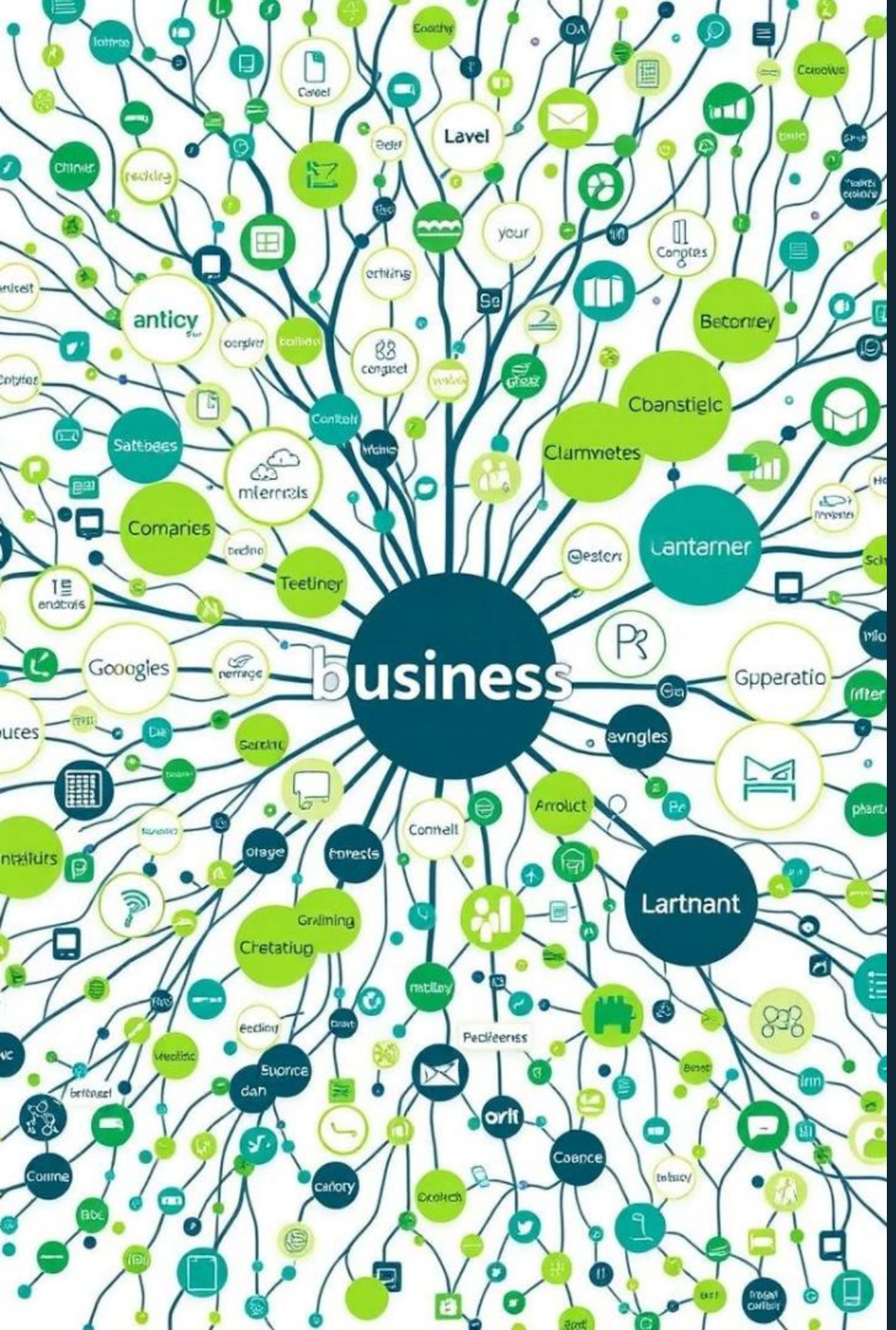
Recommendation Systems

Providing personalized recommendations based on user connections.



Conclusion and Key Takeaways

Graphs provide a powerful framework for representing relationships and connections. Understanding their terminology, types, representations, and traversal algorithms is crucial for solving complex problems in various fields.



Graphs in Business and Finance

Customer Relationship Management (CRM)

Graphs can be used to map customer interactions and relationships, helping businesses understand customer behavior and tailor marketing strategies.

Fraud Detection

Graphs can detect unusual patterns in financial transactions, helping to identify and prevent fraudulent activity.

Market Analysis

Graphs can help businesses analyze market trends, identify competitors, and understand the competitive landscape.

Risk Management

Graphs can be used to model and assess risk in financial markets, helping institutions make more informed investment decisions.

Graphs in Social Networks

Friend Recommendations

Graphs can be used to recommend new friends based on shared connections and interests.

Community Detection

Graphs can identify groups of people with similar interests or connections, fostering a sense of belonging and community.

Influence Analysis

Graphs can help understand the influence of individuals and groups within a network, identifying key opinion leaders and influencers.

Network Analysis

Graphs can be used to analyze the structure and dynamics of social networks, understanding how information flows and spreads within a community.

The Future of Graph Applications

The applications of graph technology are constantly evolving. As data becomes more complex and interconnected, graph applications will become increasingly important for analyzing and understanding this data. They will play a vital role in shaping our world, from improving healthcare outcomes to creating smarter cities and fostering a more connected society.

The Dijkstra Algorithm: A Step-by-Step Explanation

This presentation explores the Dijkstra algorithm, a fundamental concept in computer science used to find the shortest path between two points in a graph.

Defining the Problem: Finding the Shortest Path

Start Node

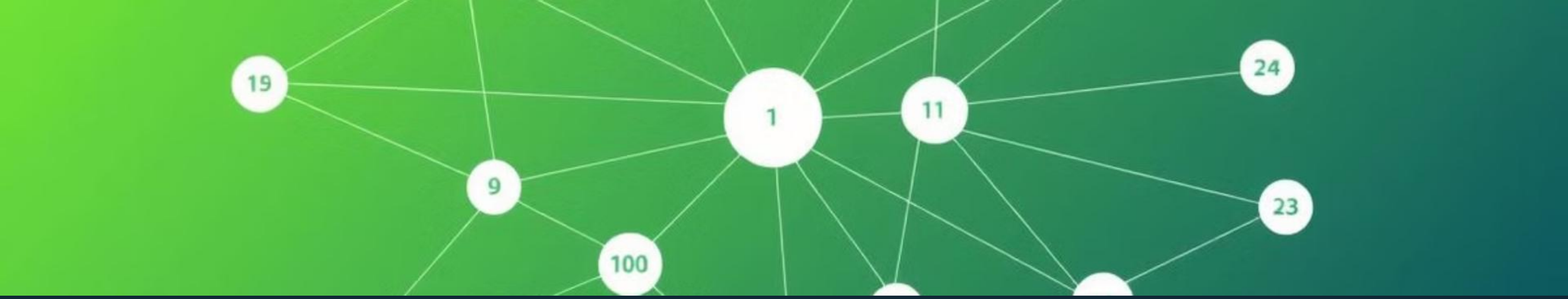
The algorithm begins at a specified start node, which serves as the origin of the path.

End Node

The goal is to determine the shortest path from the start node to a designated end node, which is also specified in the graph.

Shortest Path

The algorithm identifies the path with the lowest cumulative weight (distance or cost) among all possible routes from the start to the end node.



Key Concepts: Vertices, Edges, and Weights

Vertices

Vertices represent the points or locations in the graph.

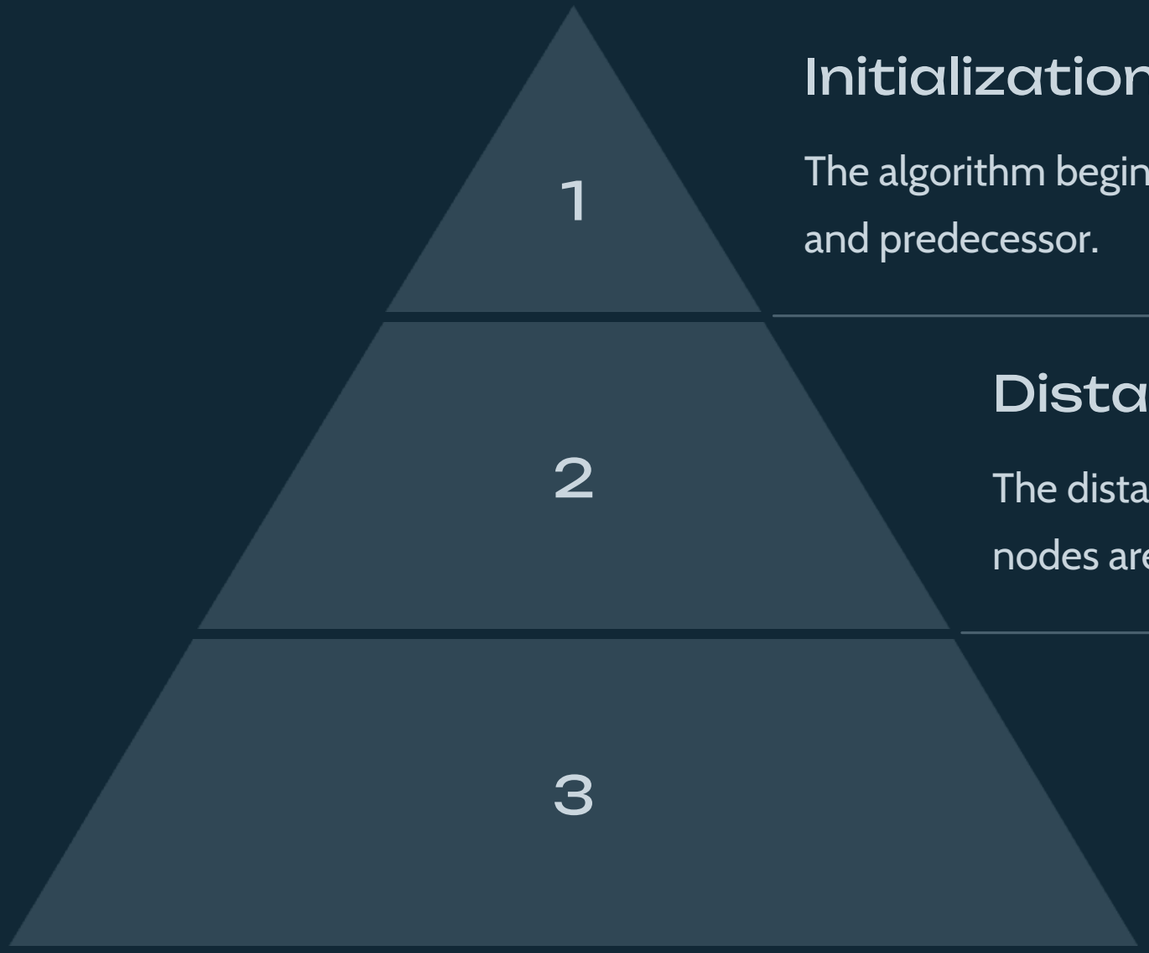
Edges

Edges represent the connections between vertices, and the direction of the edges depends on whether the graph is directed or undirected.

Weights

Weights assigned to edges indicate the distance, cost, or time required to traverse from one vertex to another.

Initializing the Algorithm



Initialization

The algorithm begins by initializing a table with three columns: vertex, distance, and predecessor.

Distance to Start Node

The distance to the start node is set to 0, and the distances to all other nodes are initially set to infinity, representing an unknown distance.

Predecessor

The predecessor column is set to null for all nodes, indicating that no path has been discovered yet.

Selecting the Minimum-Cost Vertex

1

Minimum Distance

The algorithm identifies the vertex with the minimum distance value, which is the next vertex to be processed.

2

Unvisited Vertex

The selected vertex must be an unvisited vertex, meaning that its distance has not yet been finalized.

3

Updating Distances

Once the minimum-cost vertex is selected, its neighbors are processed. The algorithm uses the distances to reach the minimum-cost vertex as a starting point to calculate distances to its neighbors.

Updating the Distance Estimates

1

Minimum-Cost Vertex

The algorithm considers each edge emanating from the selected vertex.

2

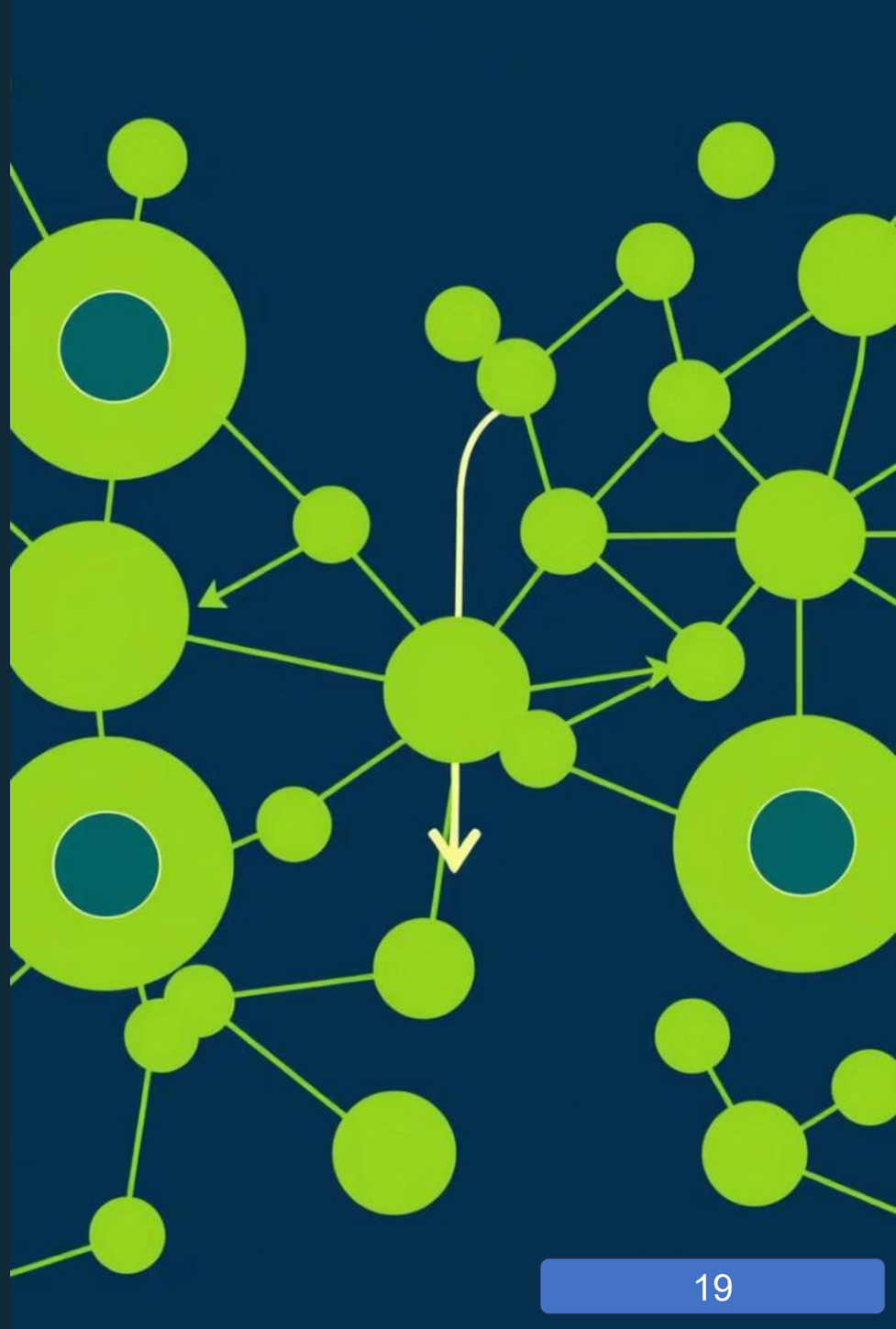
Neighbor Node

For each edge, it examines the distance to the neighboring node, adding the weight of the edge to the distance of the selected vertex.

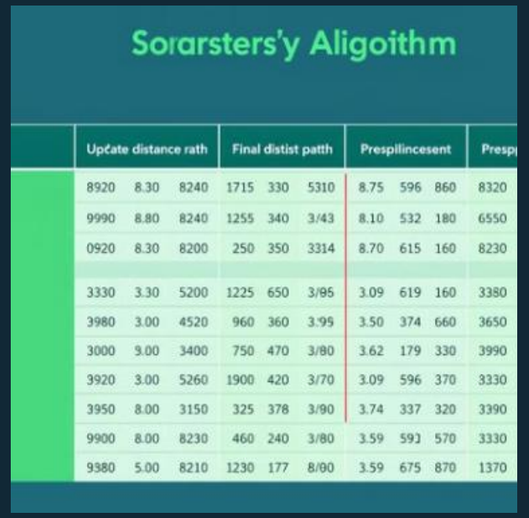
3

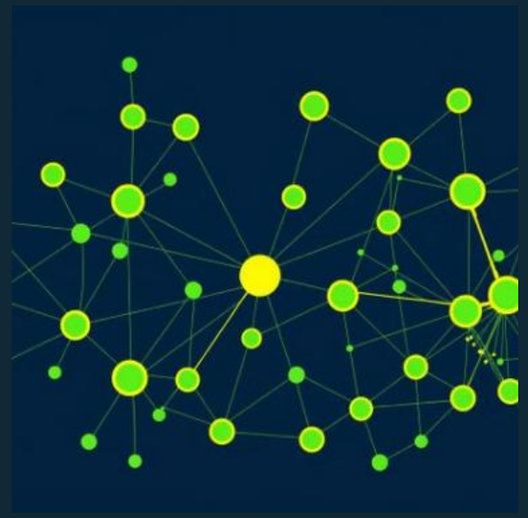
Relaxation

If the calculated distance to the neighboring node is less than the current distance in the table, the algorithm updates the distance estimate in the table.



Repeating the Process



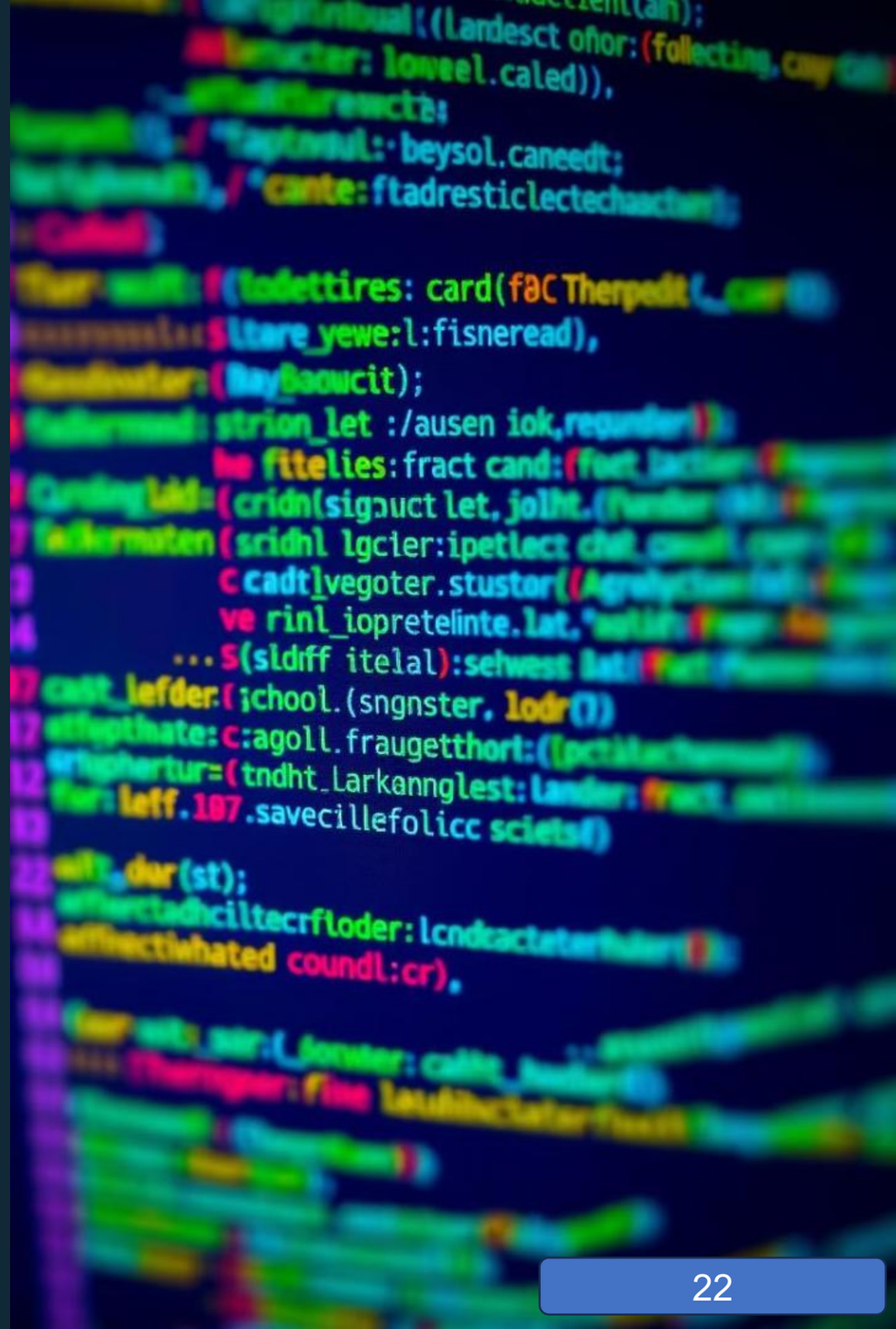


Conclusion: The Power of Dijkstra's Algorithm

Dijkstra's Algorithm is a fundamental algorithm in computer science, providing a systematic way to find the shortest path between any two nodes in a graph. Its applications are diverse, including navigation systems, network routing, and other scenarios involving optimal path planning. The algorithm's efficiency and effectiveness make it an indispensable tool in modern computing.

Dijkstra's Algorithm: Choosing the Right Data Structure

Dijkstra's Algorithm is a powerful tool for finding the shortest path between two points in a graph. Choosing the right data structure can significantly impact its efficiency.



Comparing Data Structures for Dijkstra's Algorithm

1 Array

Simple to implement but inefficient for searching and sorting.

2 Linked List

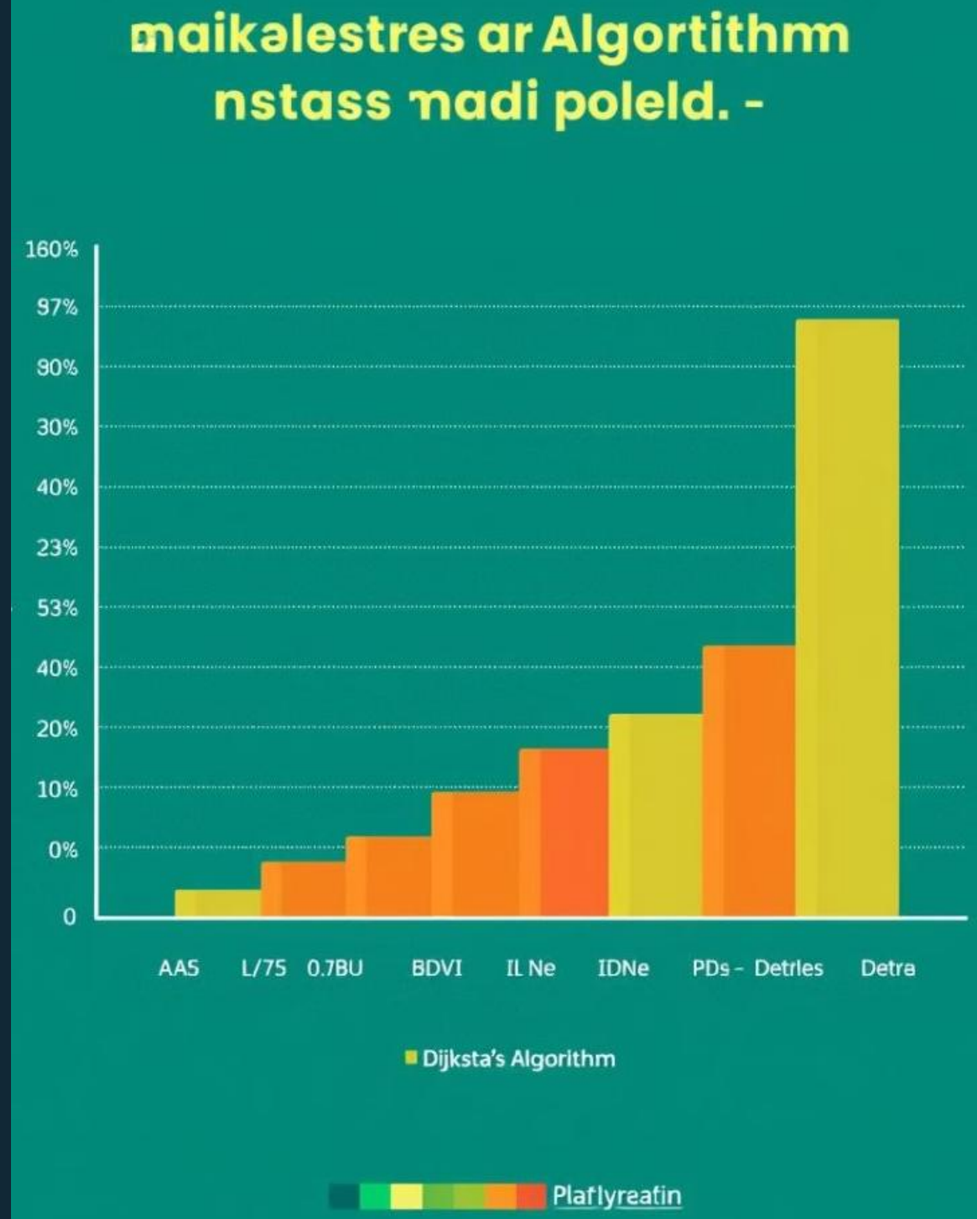
Flexible, but inefficient for random access and finding the minimum distance.

3 Binary Heap

Offers logarithmic insertion and deletion time but requires additional space for heap organization.

4 Priority Queue

A specialized data structure specifically designed for priority-based operations.



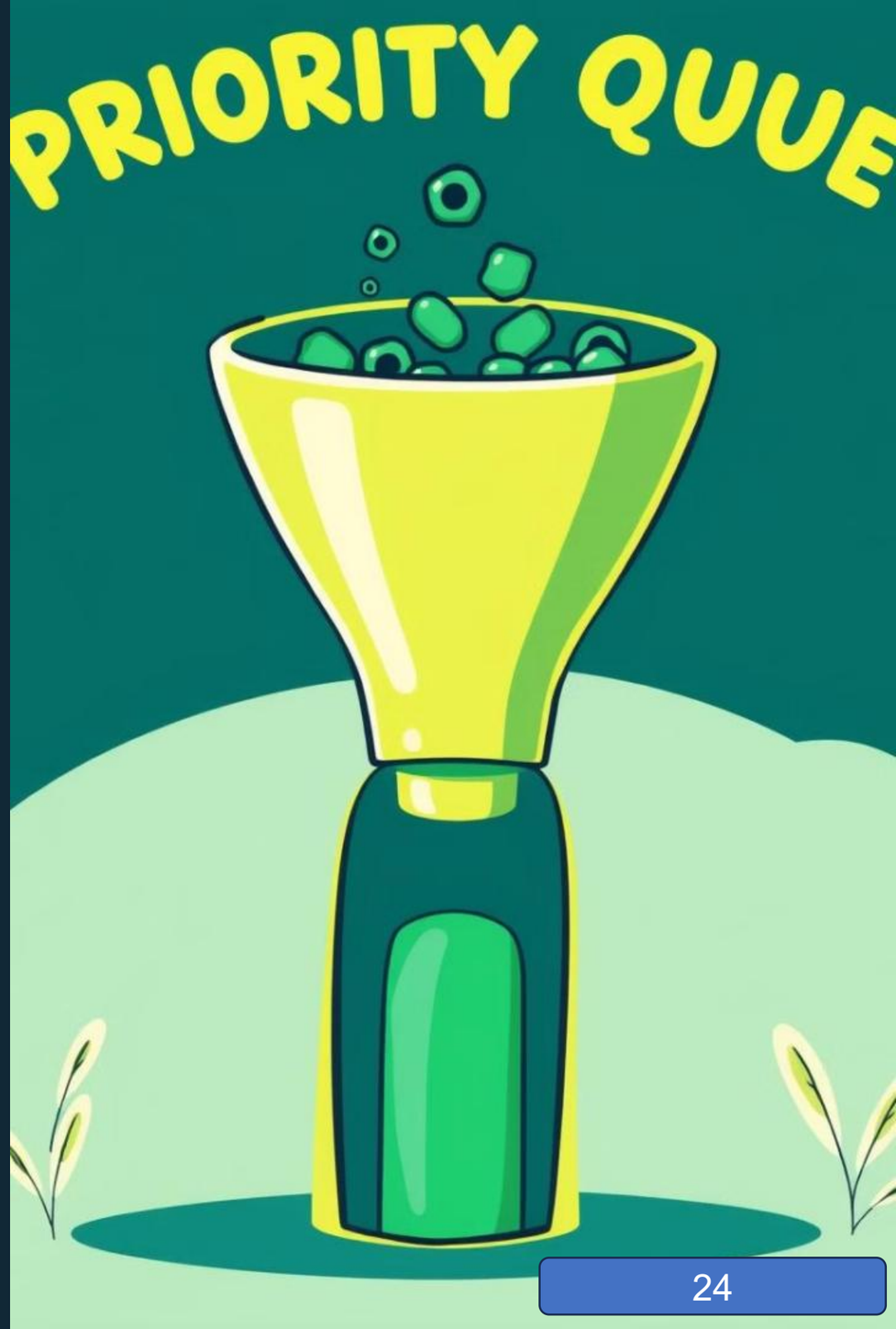
Why is the Priority Queue the Best Choice?

Efficient Selection

Priority queues enable fast retrieval of the node with the shortest distance, crucial for Dijkstra's Algorithm.

Optimized Updates

As new paths are explored, priority queues efficiently update node distances without disrupting overall structure.



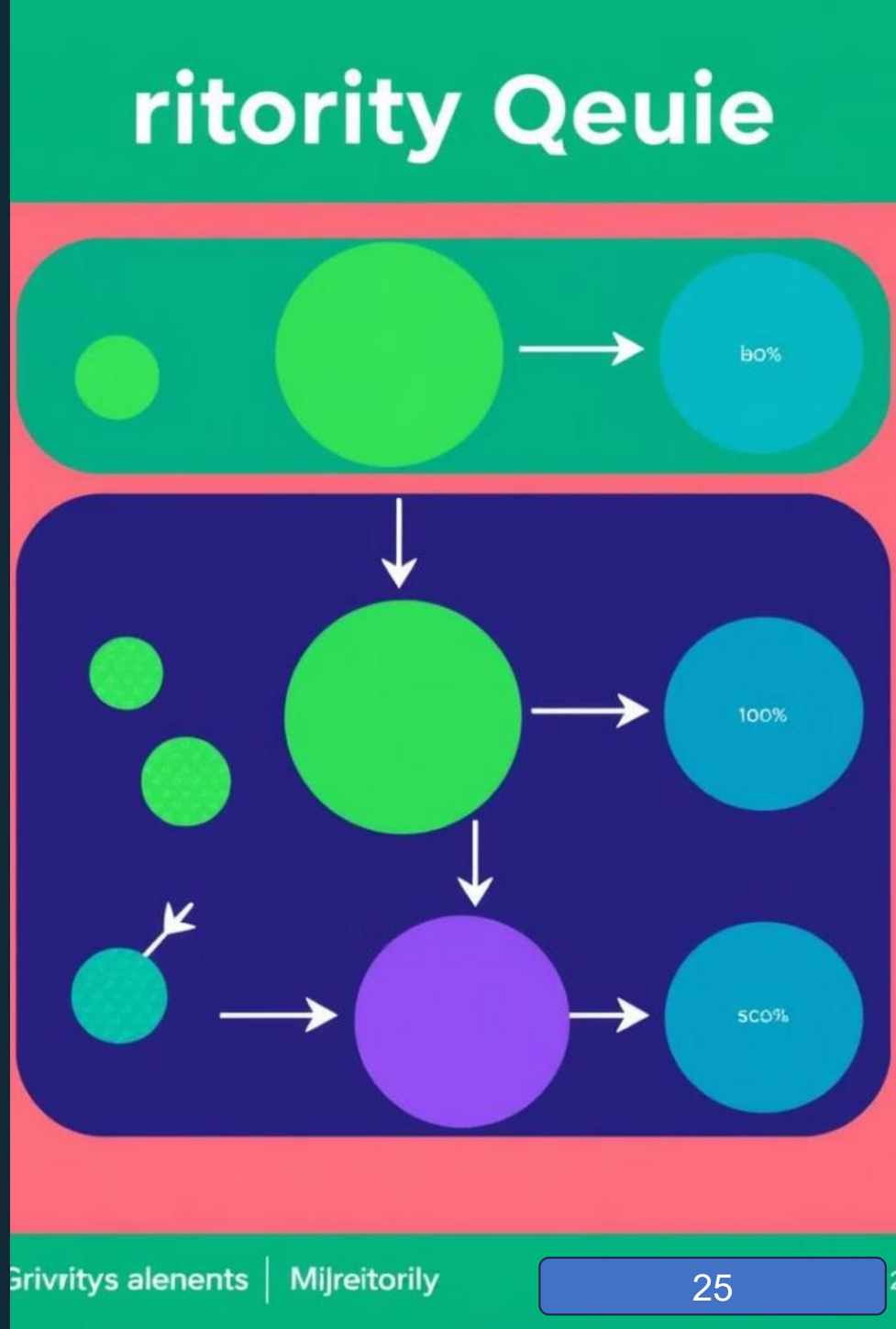
Pros of Using a Priority Queue

Speed

Priority queues provide logarithmic time complexity for key operations, enhancing algorithm efficiency.

Structure

They inherently maintain order based on priority, ensuring correct path selection during the algorithm.



Cons of Using a Priority Queue

1

Implementation Complexity

Priority queues require careful implementation, which can be complex for beginners.

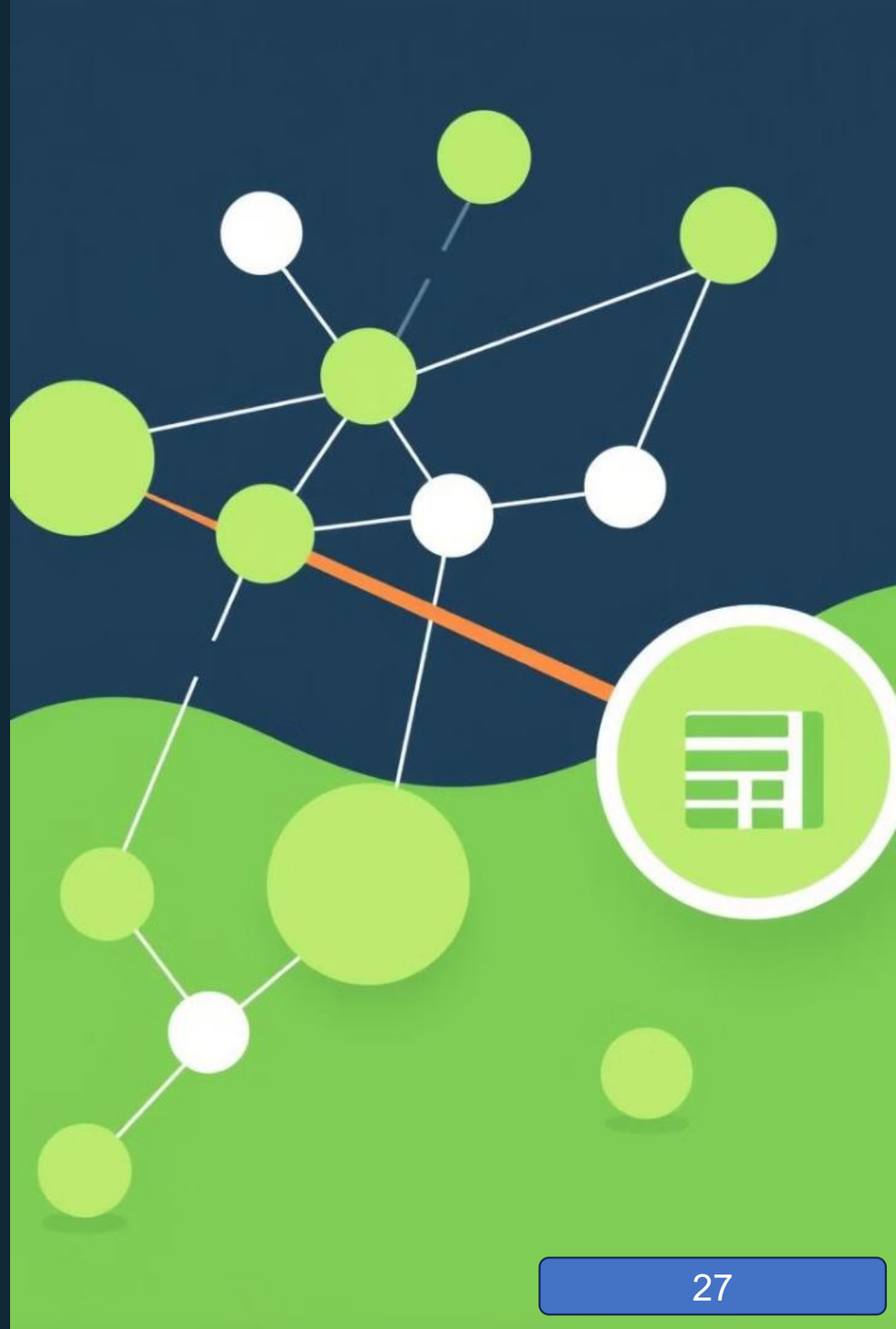
2

Space Overhead

Priority queues often have additional space requirements compared to simpler structures like arrays.

Conclusion and Key Takeaways

While other data structures can be used with Dijkstra's Algorithm, the priority queue stands out as the most efficient and effective choice due to its specialized design and performance characteristics. Choosing the right data structure is crucial for optimizing the algorithm's speed and overall efficiency.



Solving a Graph Example Using Dijkstra's Algorithm

This presentation will demonstrate the step-by-step process of applying Dijkstra's algorithm to find the shortest path between two nodes in a graph. We will break down each step in detail, using a visual example to illustrate the algorithm's logic.

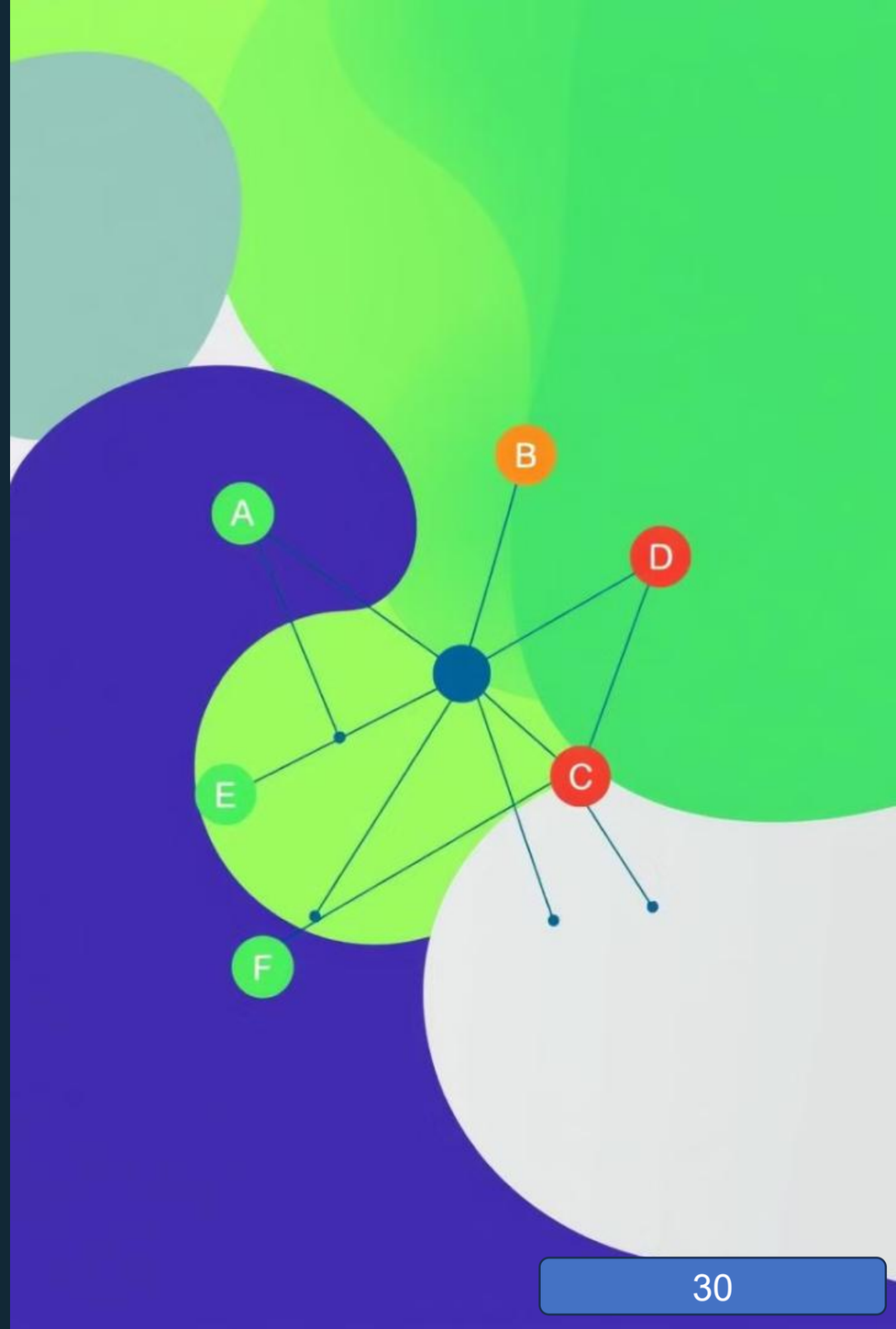
Introduction to the Graph Example

Let's consider a simple graph with six nodes: A, B, C, D, E, and F. Each node represents a location, and the edges represent connections between them. The numbers on the edges represent the distances between the nodes.

Our goal is to find the shortest path from node A to node F using Dijkstra's algorithm. This algorithm is a common method for solving single-source shortest path problems in weighted graphs.

Explaining the Graph Structure

Edge	Distance
A - B	1
A - C	2
B - C	3
B - D	4
C - D	5
C - E	6
D - F	7
E - F	8



Step 1: Initializing the Algorithm

1

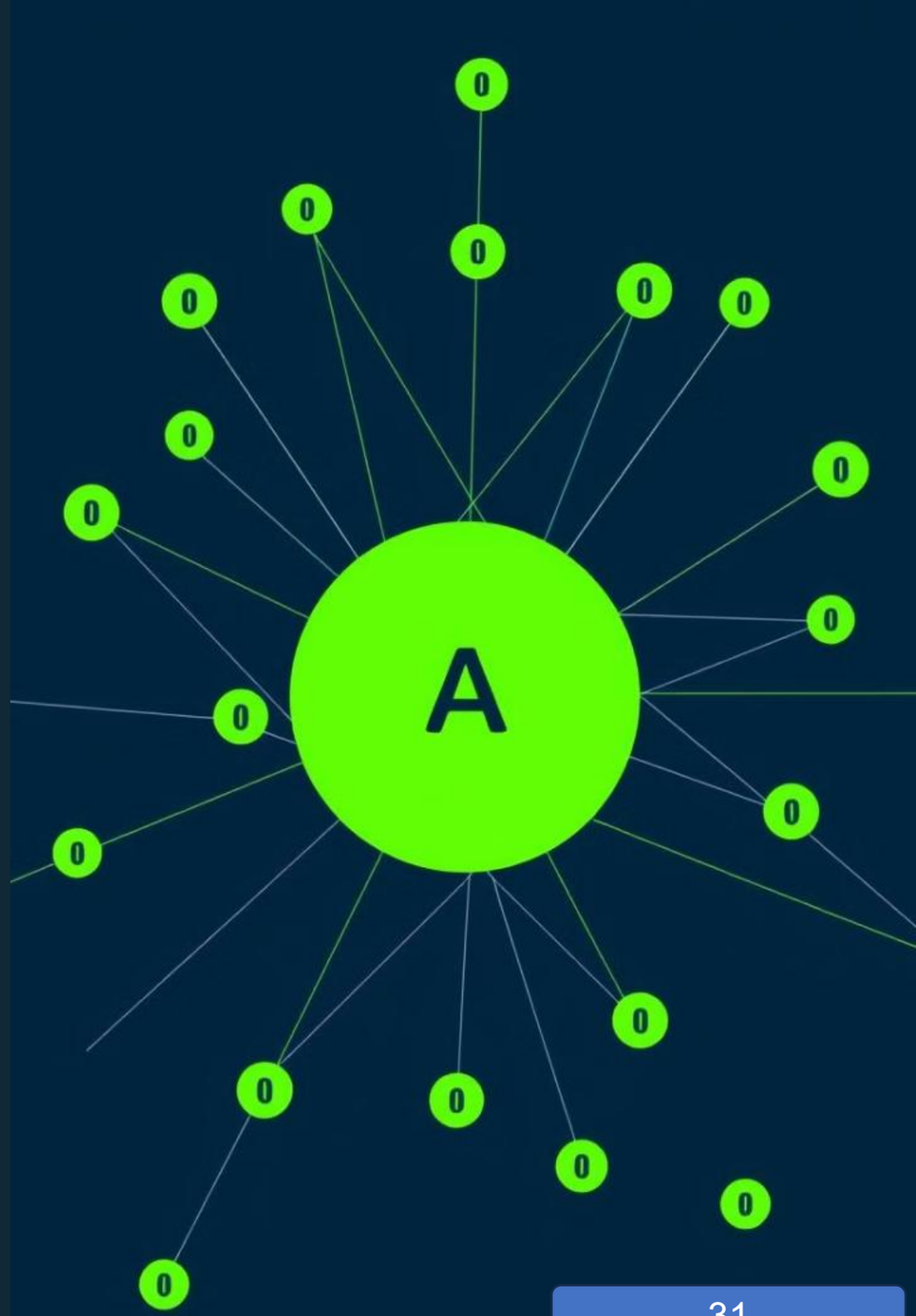
1. Initialization

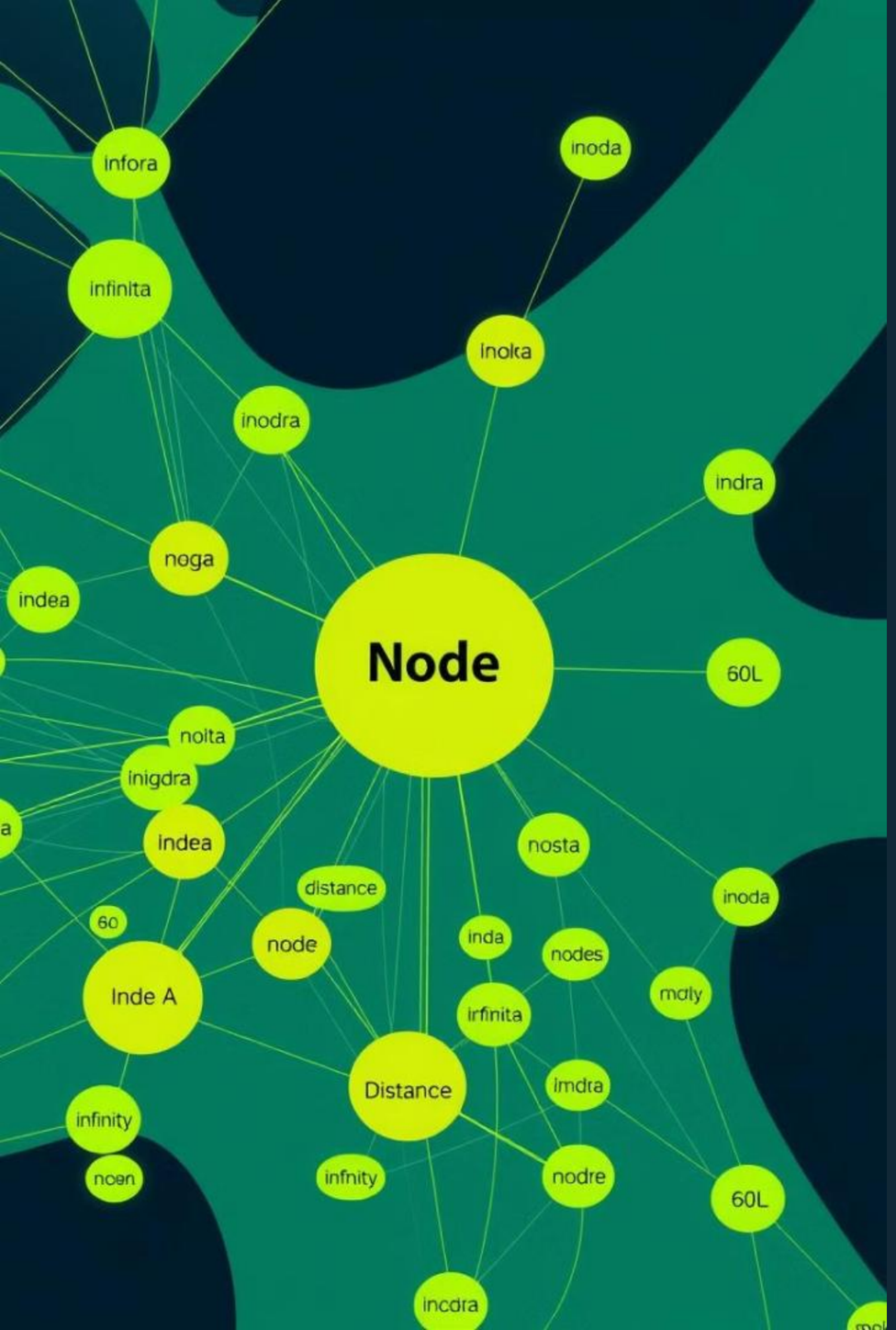
We initialize a distance table. Node A is set as the starting node, and its distance to itself is 0. The distances from A to all other nodes are initially set to infinity.

2

2. Marking Nodes

We mark all nodes as unvisited. This indicates that we haven't yet determined the shortest path from A to these nodes.





Step 2: Selecting the Starting Node

We start with node A, as it's our starting point. We mark node A as visited, meaning that we've explored its immediate connections and determined the shortest paths from A to its neighbors.

Step 3: Updating Distances and Visiting Nodes

Update Distances

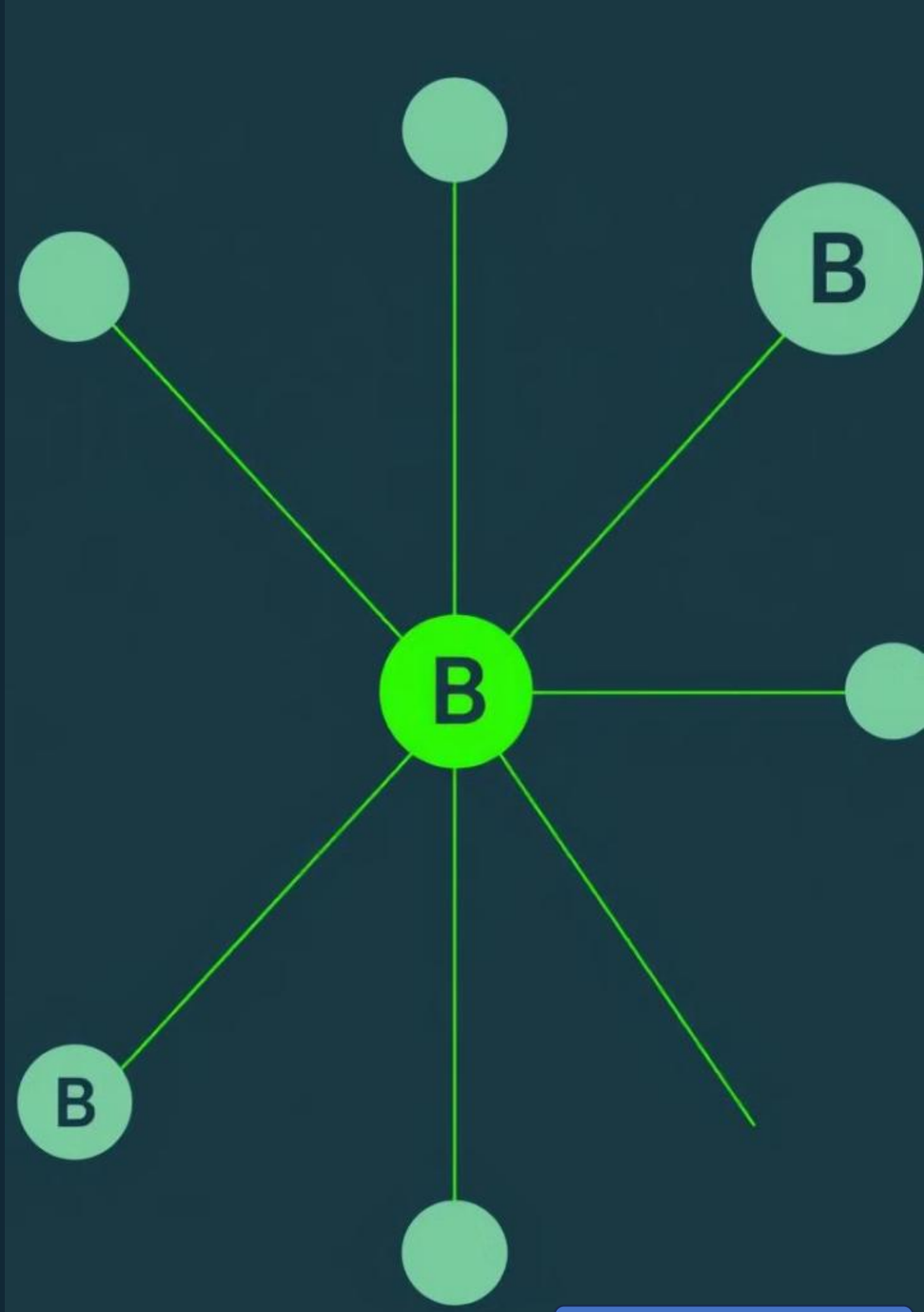
We examine the edges connected to node A. The edge connecting A to B has a weight of 1, so we update the distance from A to B to 1. Similarly, the edge connecting A to C has a weight of 2, so we update the distance from A to C to 2.

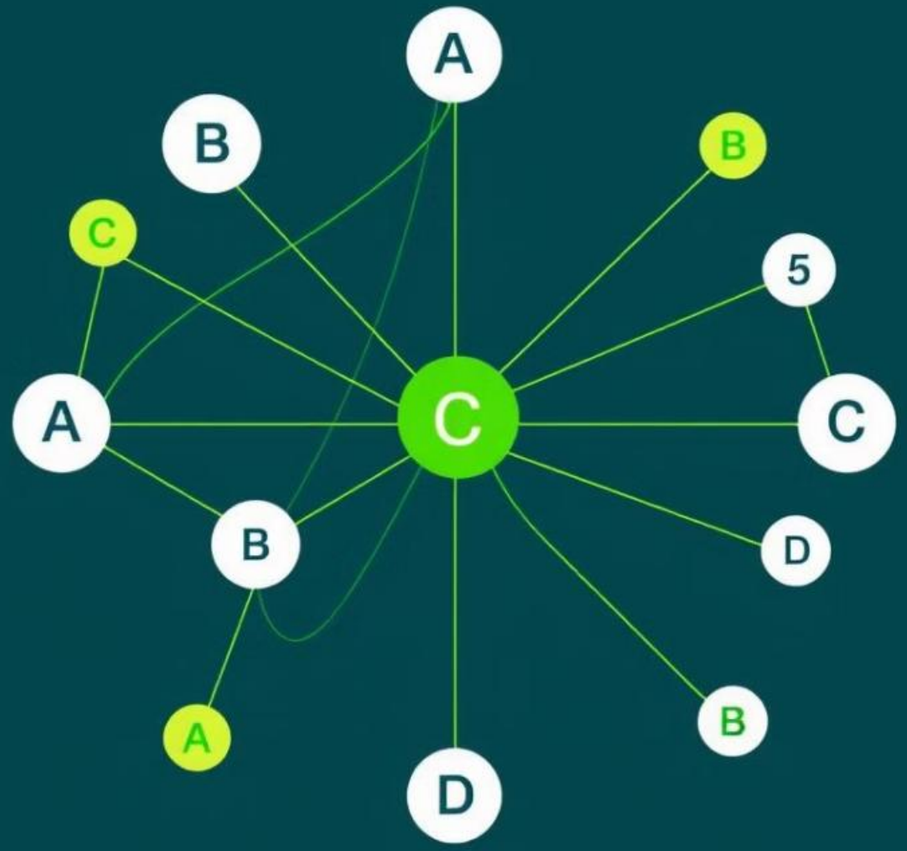
Mark as Visited

We mark nodes B and C as visited. This means we've explored all the edges connected to A and found the shortest paths from A to its immediate neighbors.

Step 4: Selecting the Next Unvisited Node

Next, we look for the unvisited node with the smallest distance from A. In this case, it's node B with a distance of 1. We mark node B as visited.





Step 5: Repeating the Process

We repeat steps 3 and 4 for node B. We examine its edges and update the distances to its unvisited neighbors: D (distance 5). We mark node D as visited. This process continues for all unvisited nodes until we reach the target node.

Visualizing the Step-by-Step Solution

1

1. Visiting Node A

The algorithm starts at node A, identifying the shortest paths to its immediate neighbors.

2

2. Visiting Node B

The algorithm moves to node B, updating distances to its unvisited neighbors.

3

3. Visiting Node C

The algorithm continues, exploring node C and adjusting distances.

4

4. Visiting Node D

The algorithm visits node D, and the process continues until the shortest path to F is found.

5

5. Visiting Node E

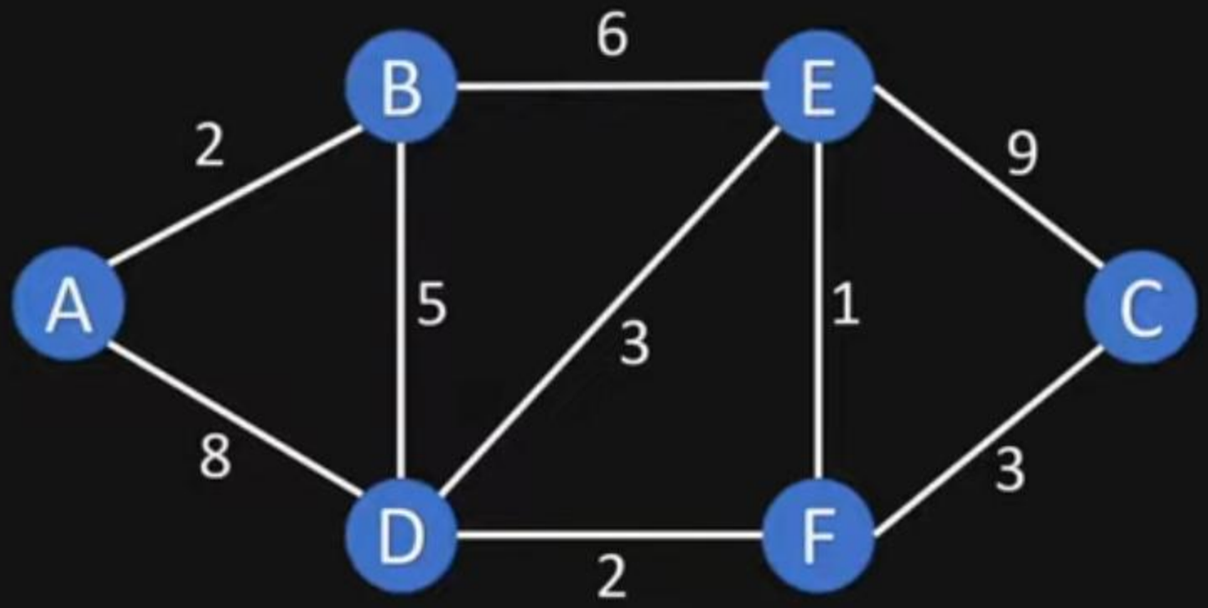
The algorithm reaches node E, and finally identifies the shortest path to F.

6

6. Reaching Node F

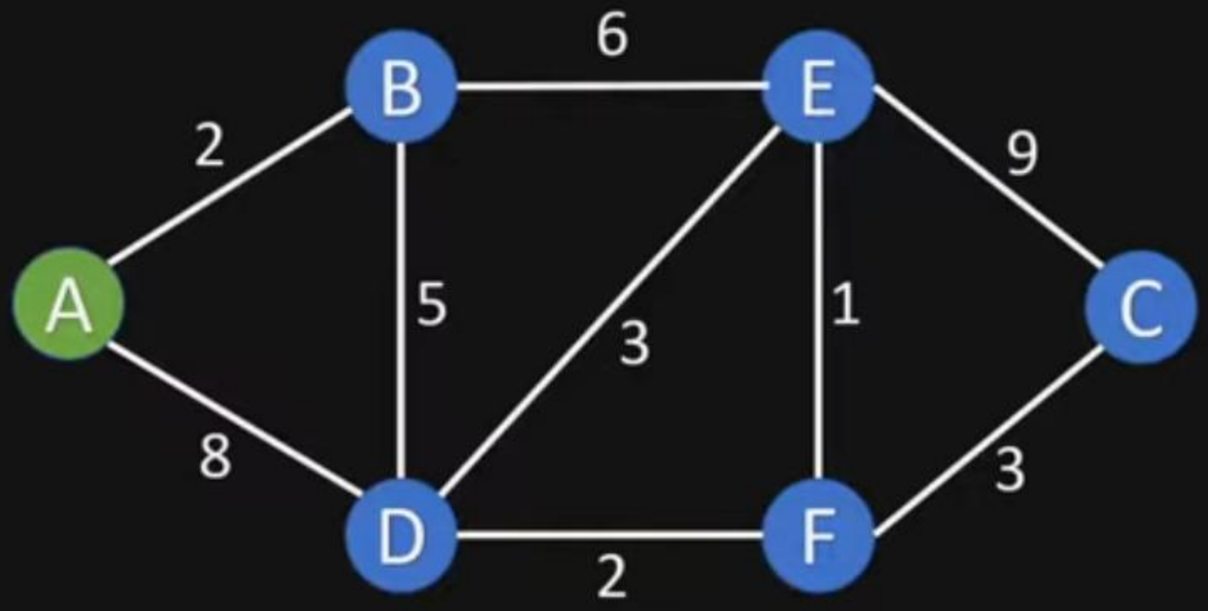
The algorithm reaches node F, marking the end of the process and indicating the shortest path.

Dijkstra's Shortest Path Algorithm



- Shortest path from a fixed node to every other node
- e.g. Cities and routes between them

2. Assign to all nodes a tentative distance value

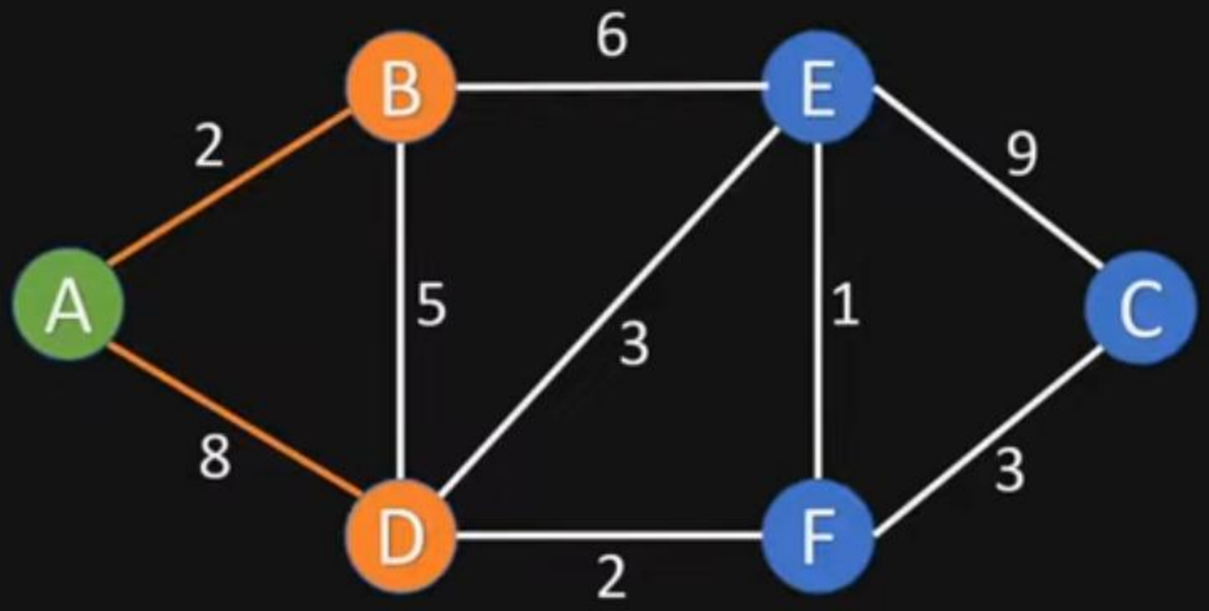


Node	Shortest Distance	Previous Node
A	0	
B	∞	
C	∞	
D	∞	
E	∞	
F	∞	

Visited Nodes: []

Unvisited Nodes: [A, B, C, D, E, F]

3. For the current node calculate the distance to all unvisited neighbours



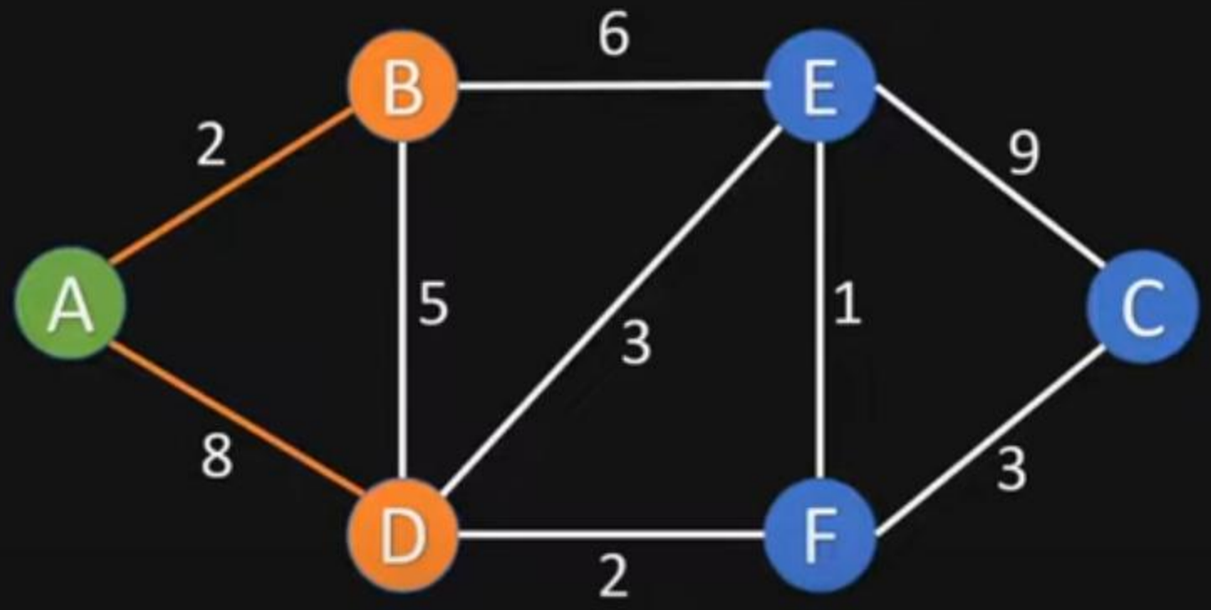
Node	Shortest Distance	Previous Node
A	0	
B	∞	
C	∞	
D	∞	
E	∞	
F	∞	

Visited Nodes: []

Unvisited Nodes: [A, B, C, D, E, F]

3. For the current node calculate the distance to all unvisited neighbours

3.1. Update shortest distance, if new distance is shorter than old distance



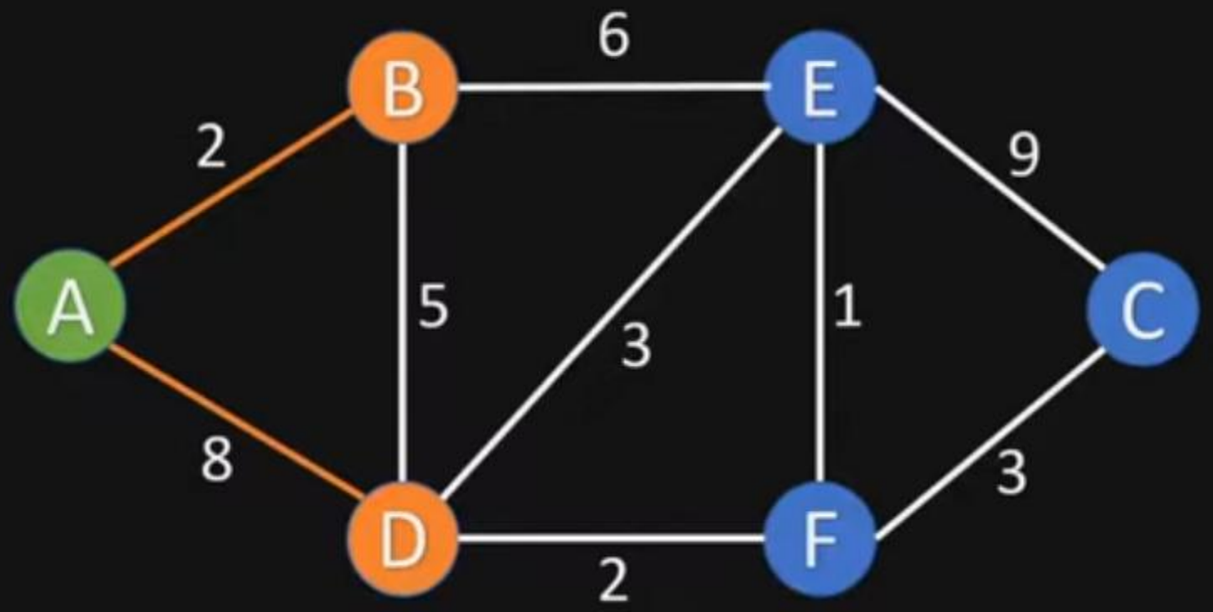
Node	Shortest Distance	Previous Node
A	0	
B	**2**	A
C	∞	
D	∞	
E	∞	
F	∞	

Visited Nodes: []

Unvisited Nodes: [A, B, C, D, E, F]

3. For the current node calculate the distance to all unvisited neighbours

3.1. Update shortest distance, if new distance is shorter than old distance

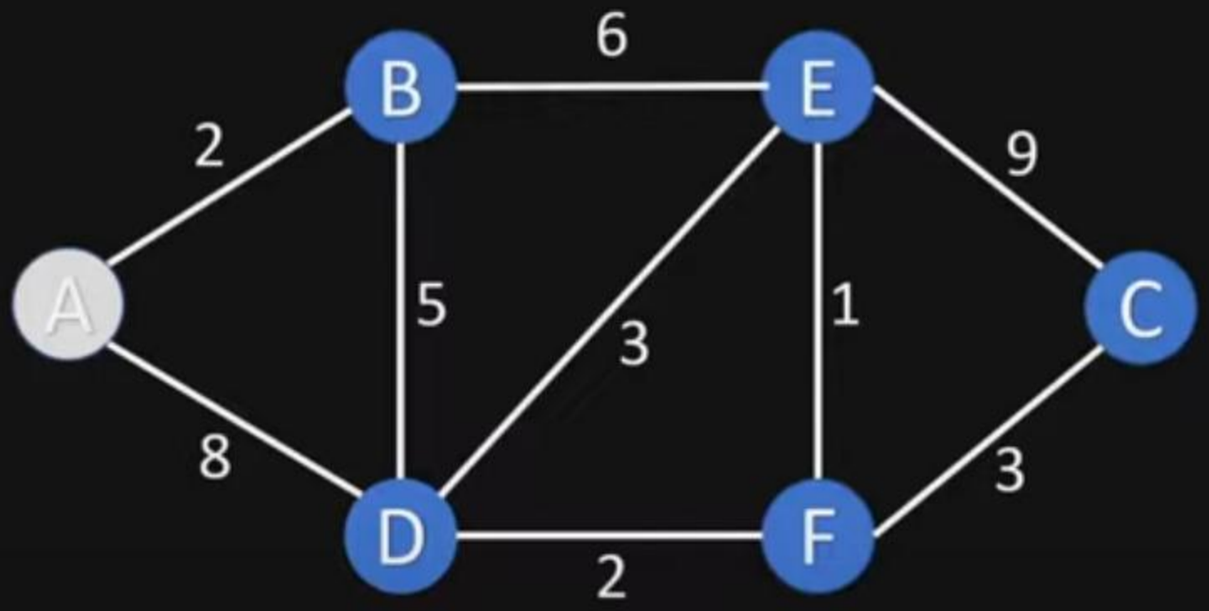


Node	Shortest Distance	Previous Node
A	0	
B	**2**	A
C	∞	
D	**8**	A
E	∞	
F	∞	

Visited Nodes: []

Unvisited Nodes: [A, B, C, D, E, F]

4. Mark current node as visited

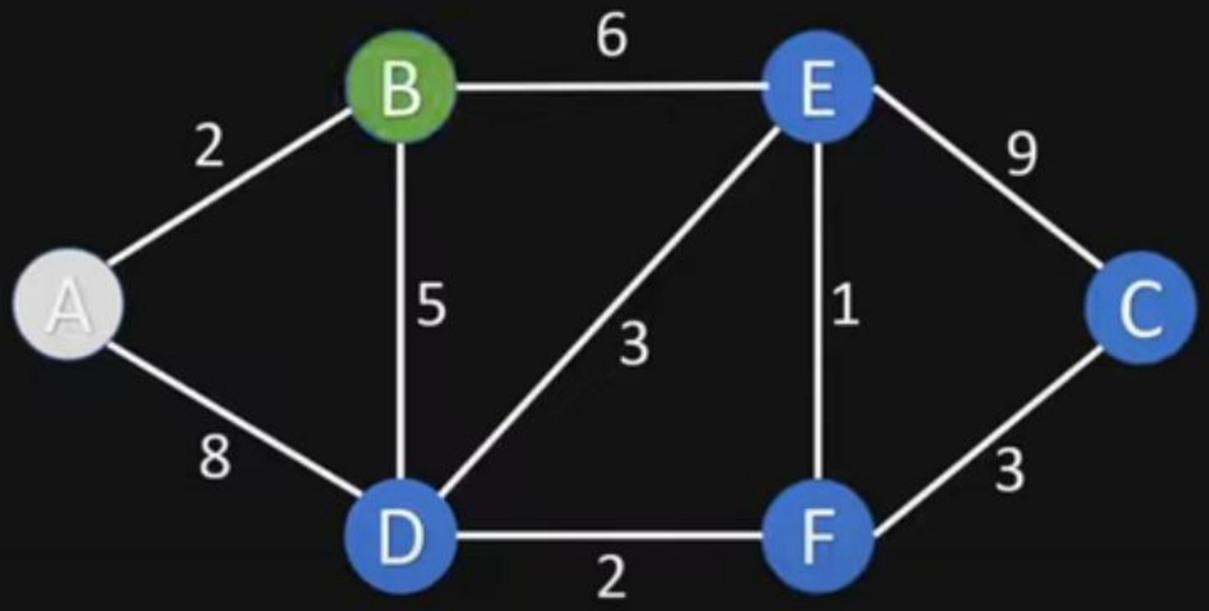


Node	Shortest Distance	Previous Node
A	0	
B	2	A
C	∞	
D	8	A
E	∞	
F	∞	

Visited Nodes: [A]

Unvisited Nodes: [B, C, D, E, F]

5. Choose new current node from unvisited nodes with minimal distance

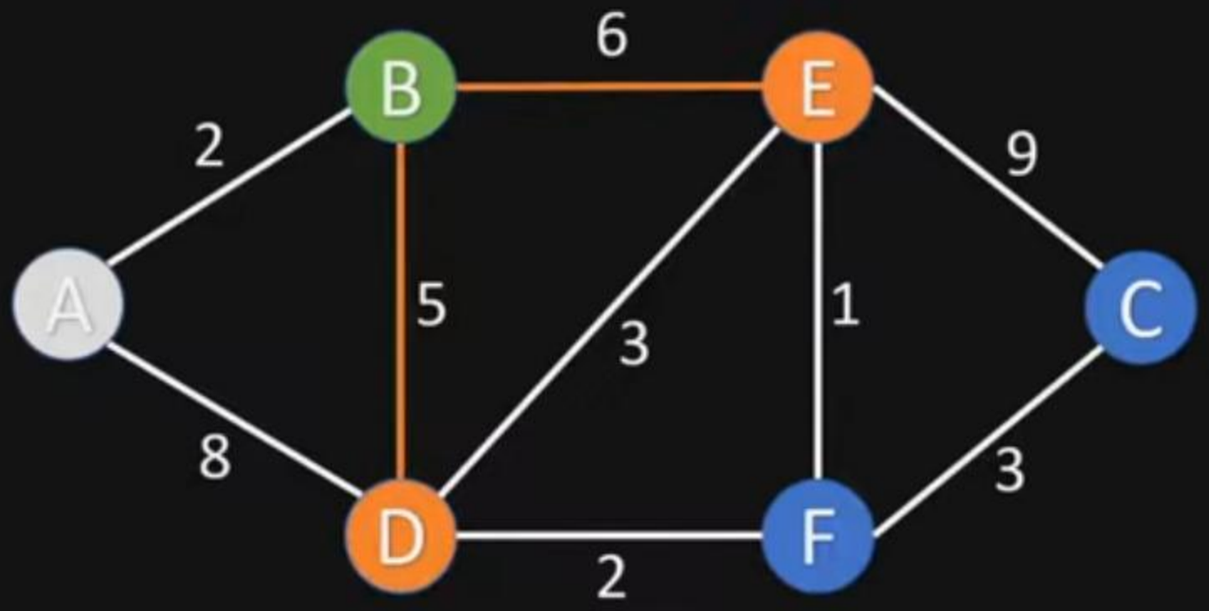


Node	Shortest Distance	Previous Node
A	0	
B	2	A
C	∞	
D	8	A
E	∞	
F	∞	

Visited Nodes: [A]

Unvisited Nodes: [B, C, D, E, F]

3. For the current node calculate the distance to all unvisited neighbours

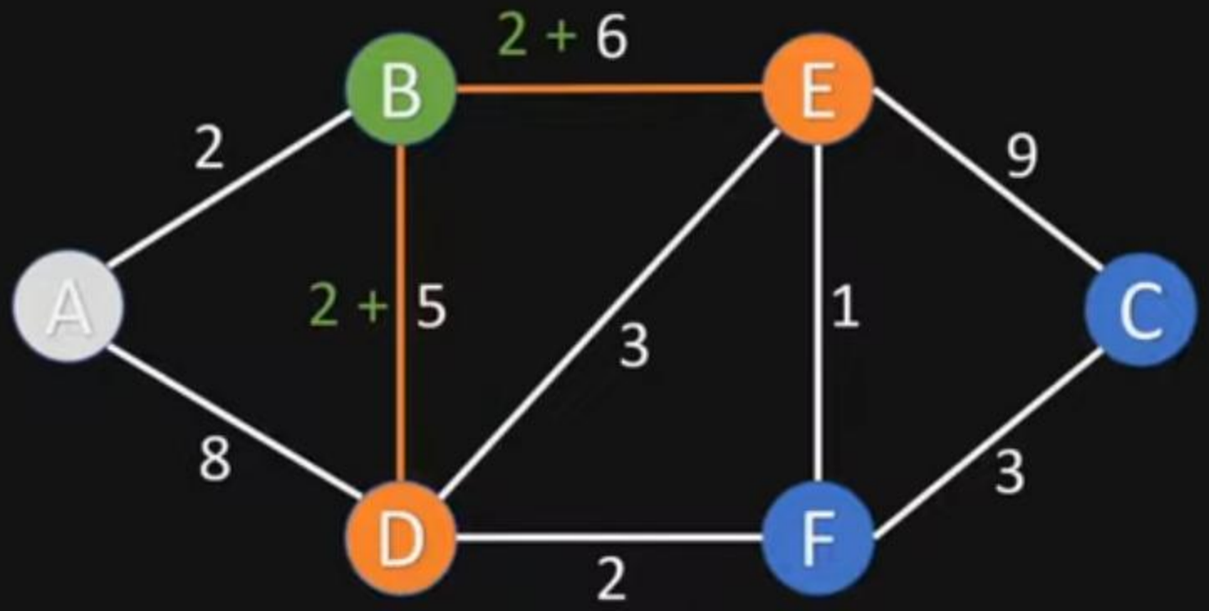


Node	Shortest Distance	Previous Node
A	0	
B	2	A
C	∞	
D	8	A
E	∞	
F	∞	

Visited Nodes: [A]

Unvisited Nodes: [B, C, D, E, F]

3. For the current node calculate the distance to all unvisited neighbours



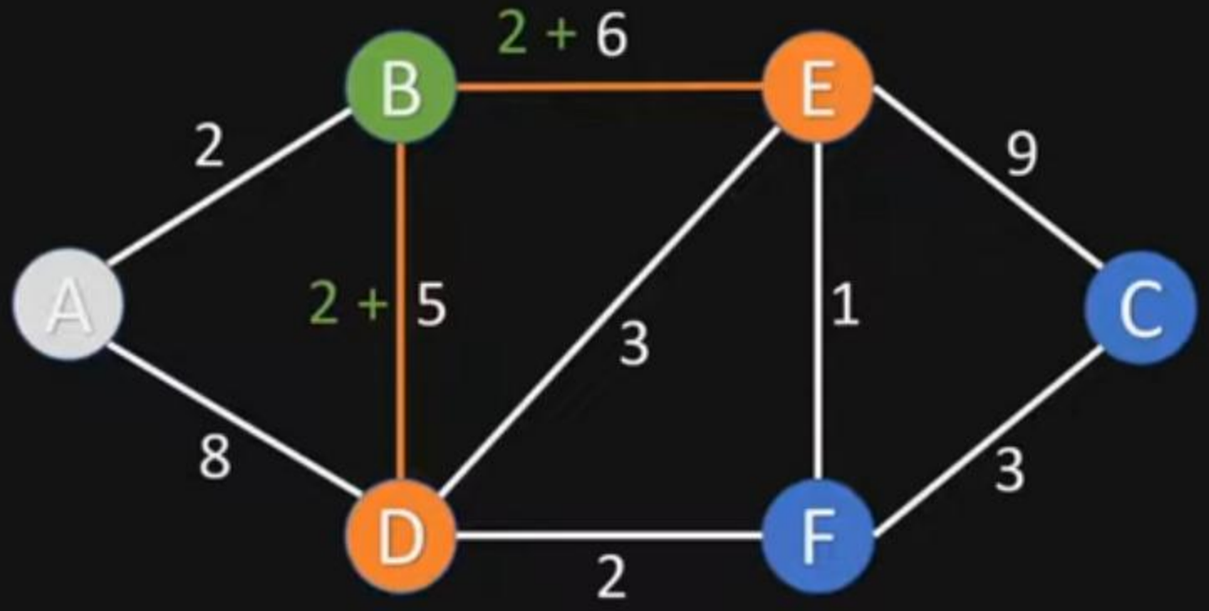
Node	Shortest Distance	Previous Node
A	0	
B	2	A
C	∞	
D	8	A
E	∞	
F	∞	

Visited Nodes: [A]

Unvisited Nodes: [B, C, D, E, F]

3. For the current node calculate the distance to all unvisited neighbours

3.1. Update shortest distance, if new distance is shorter than old distance



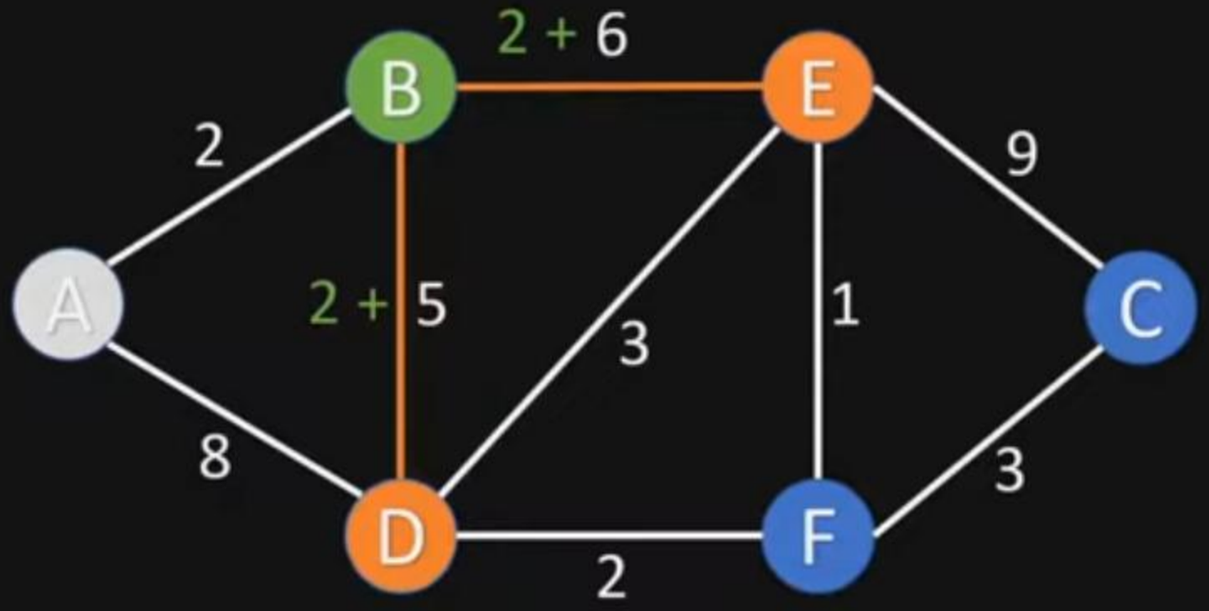
Node	Shortest Distance	Previous Node
A	0	
B	**2**	A
C	∞	
D	**8**	A
E	**8**	B
F	∞	

Visited Nodes: [A]

Unvisited Nodes: [B, C, D, E, F]

3. For the current node calculate the distance to all unvisited neighbours

3.1. Update shortest distance, if new distance is shorter than old distance

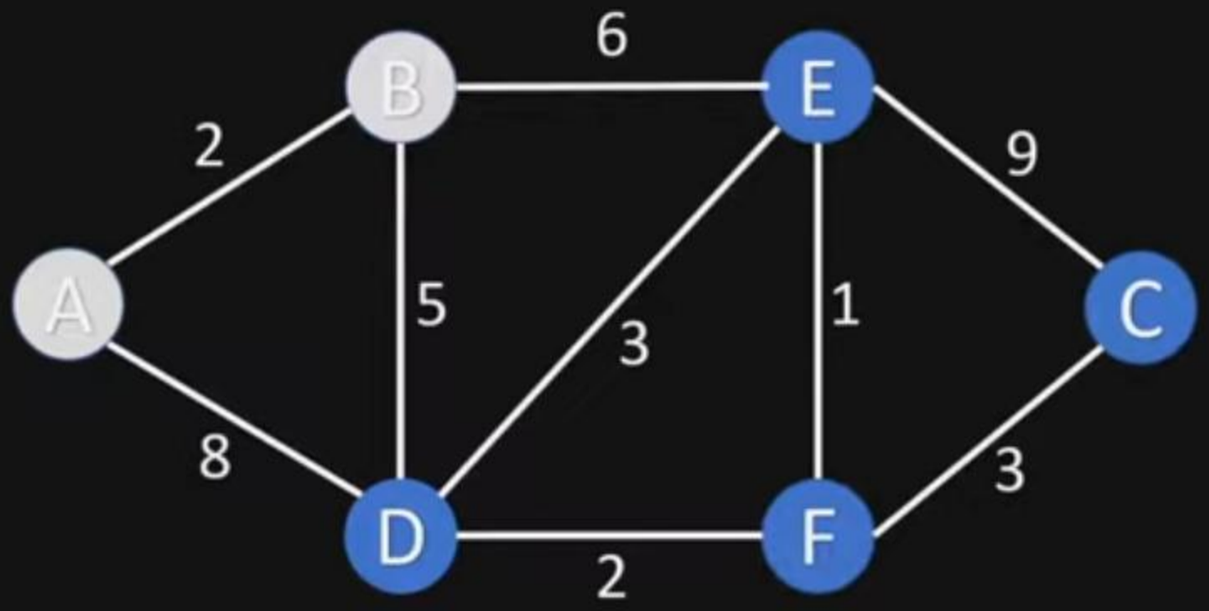


Node	Shortest Distance	Previous Node
A	0	
B	**2**	A
C	∞	
D	**7**	B
E	**8**	B
F	∞	

Visited Nodes: [A]

Unvisited Nodes: [B, C, D, E, F]

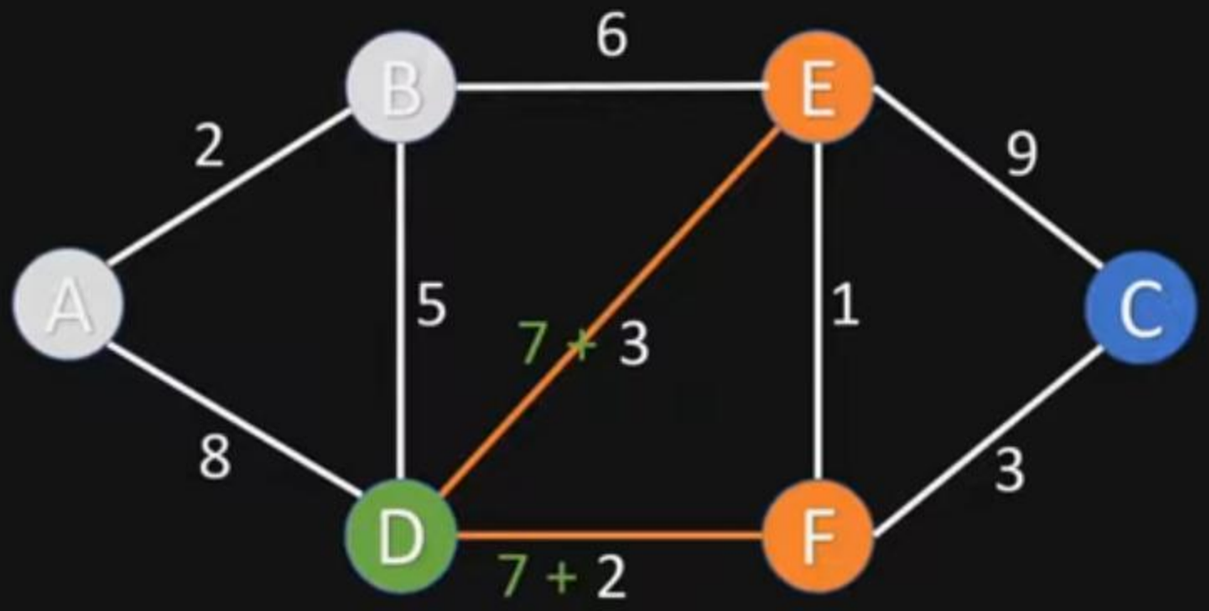
4. Mark current node as visited



Node	Shortest Distance	Previous Node
A	0	
B	2	A
C	∞	
D	7	B
E	8	B
F	∞	

Visited Nodes: [A, B] Unvisited Nodes: [C, D, E, F]

3. For the current node calculate the distance to all unvisited neighbours

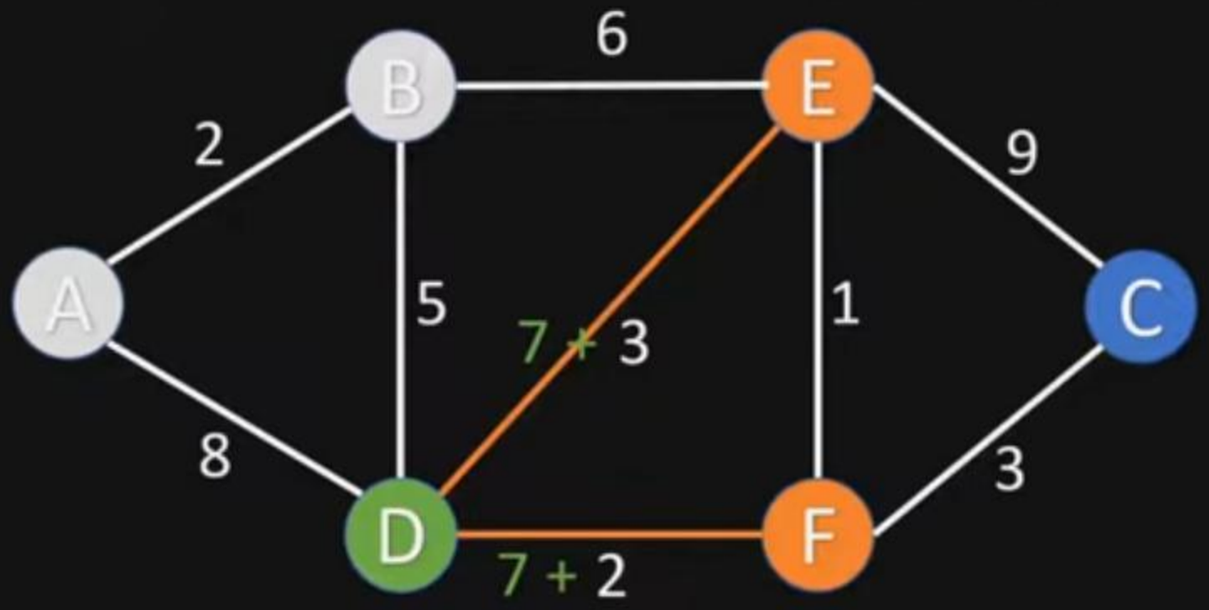


Node	Shortest Distance	Previous Node
A	0	
B	2	A
C	∞	
D	7	B
E	8	B
F	∞	

Visited Nodes: [A, B] Unvisited Nodes: [C, D, E, F]

3. For the current node calculate the distance to all unvisited neighbours

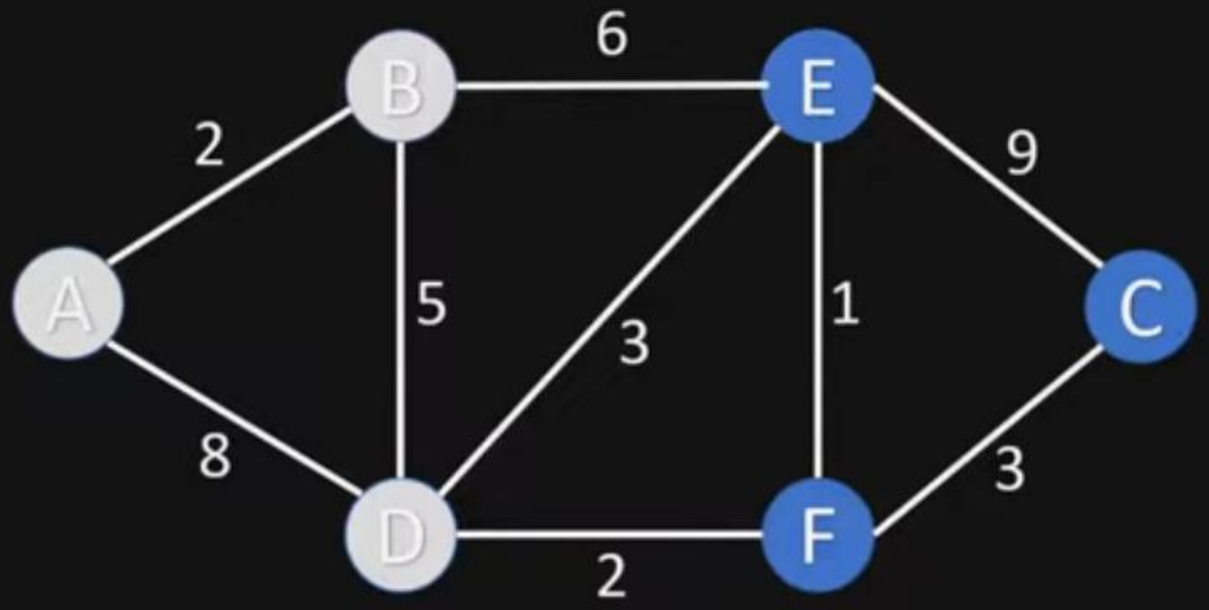
3.1. Update shortest distance, if new distance is shorter than old distance



Node	Shortest Distance	Previous Node
A	0	
B	**2**	A
C	**∞**	
D	**7**	B
E	**8**	B
F	**9**	D

Visited Nodes: [A, B] Unvisited Nodes: [C, D, E, F]

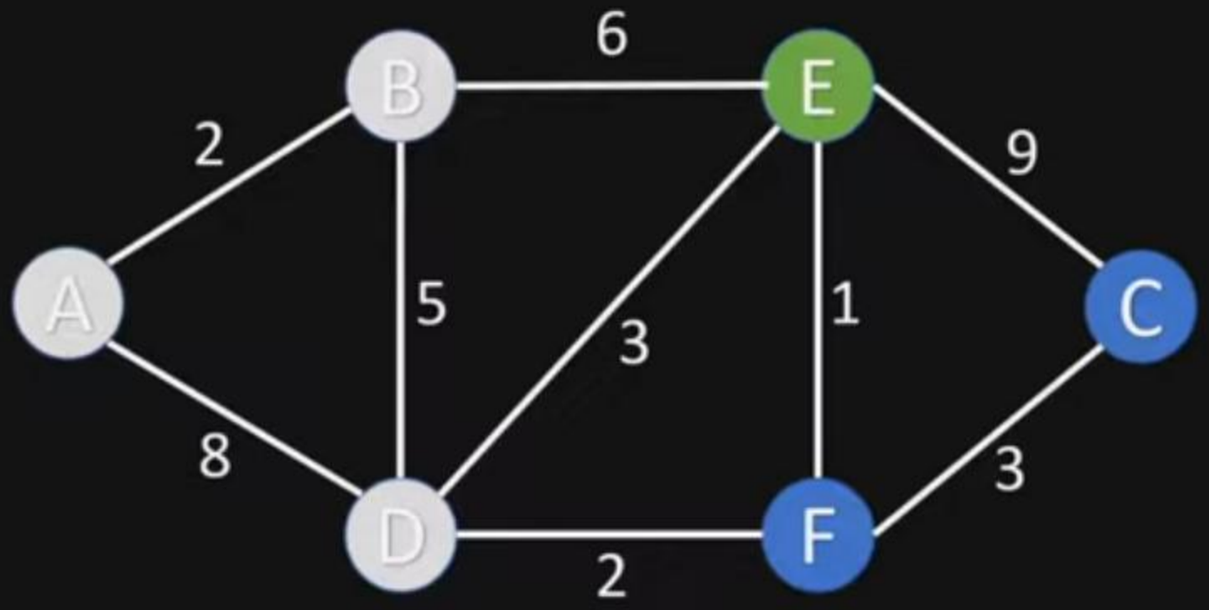
4. Mark current node as visited



Node	Shortest Distance	Previous Node
A	0	
B	**2**	A
C	**∞**	
D	**7**	B
E	**8**	B
F	**9**	D

Visited Nodes: [A, B, D] Unvisited Nodes: [C, D, E, F]

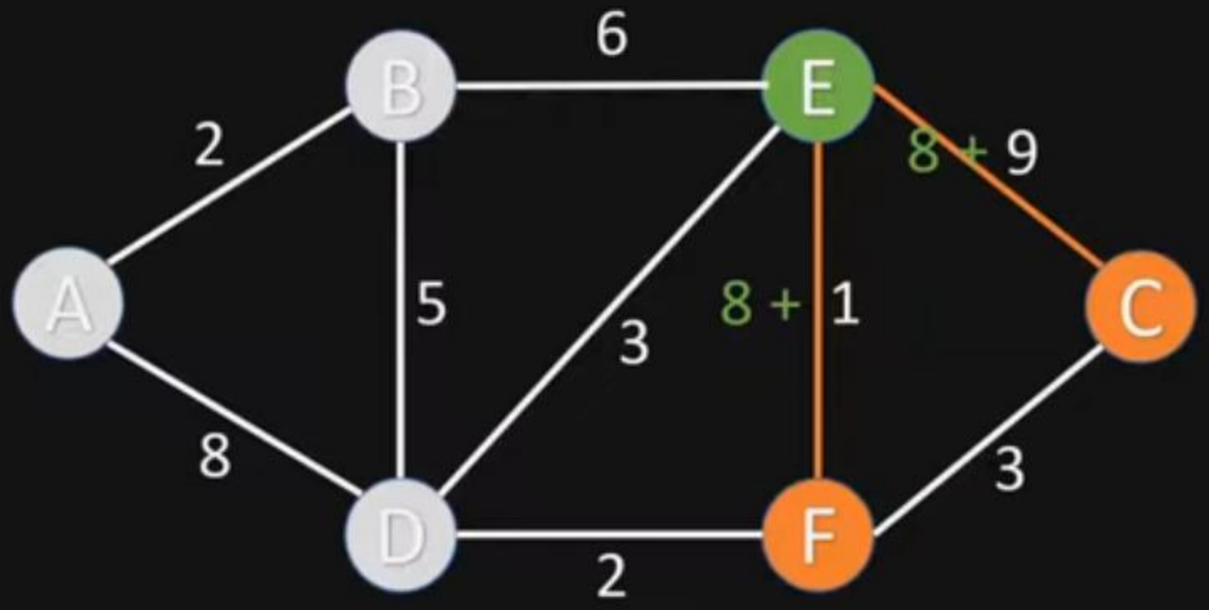
5. Choose new current node from unvisited nodes with minimal distance



Node	Shortest Distance	Previous Node
A	0	
B	2	A
C	∞	
D	7	B
E	8	B
F	9	D

Visited Nodes: [A, B, D] Unvisited Nodes: [C, E, F]

3. For the current node calculate the distance to all unvisited neighbours

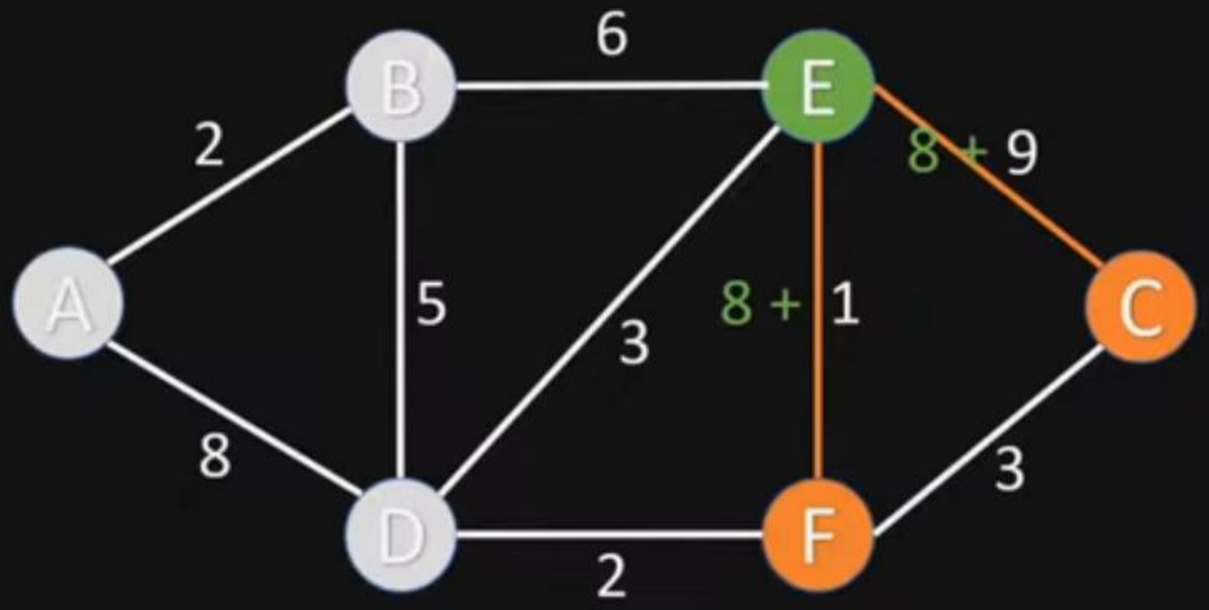


Node	Shortest Distance	Previous Node
A	0	
B	**2**	A
C	∞	
D	**7**	B
E	**8**	B
F	**9**	D

Visited Nodes: [A, B, D] Unvisited Nodes: [C, E, F]

3. For the current node calculate the distance to all unvisited neighbours

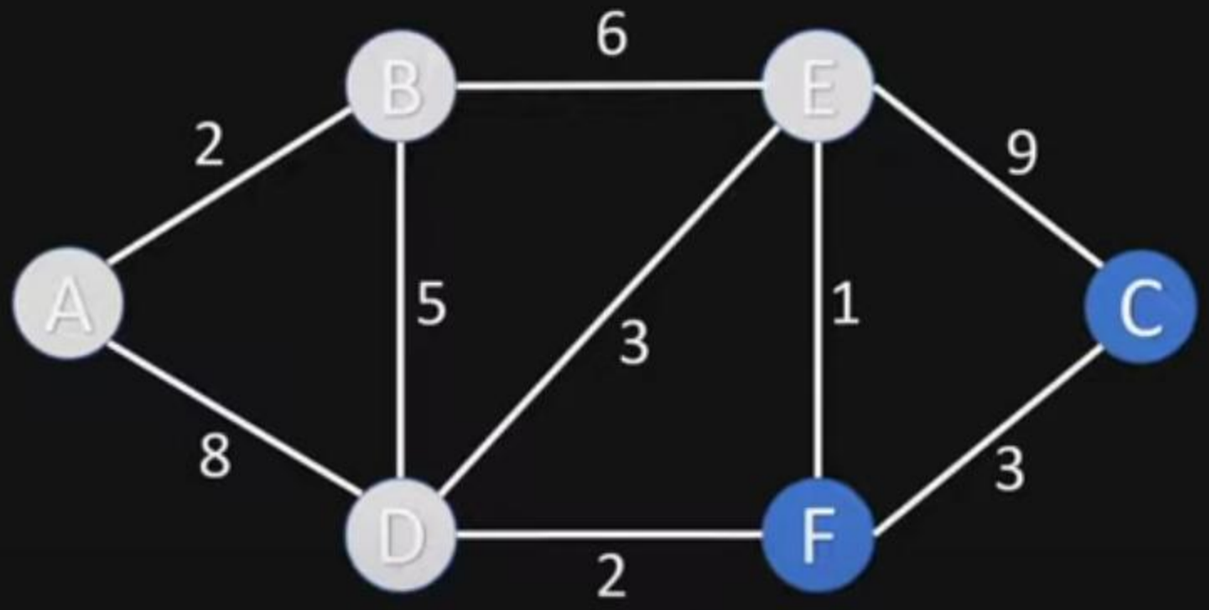
3.1. Update shortest distance, if new distance is shorter than old distance



Node	Shortest Distance	Previous Node
A	0	
B	**2**	A
C	**17**	E
D	**7**	B
E	**8**	B
F	**9**	D

Visited Nodes: [A, B, D] Unvisited Nodes: [C, E, F]

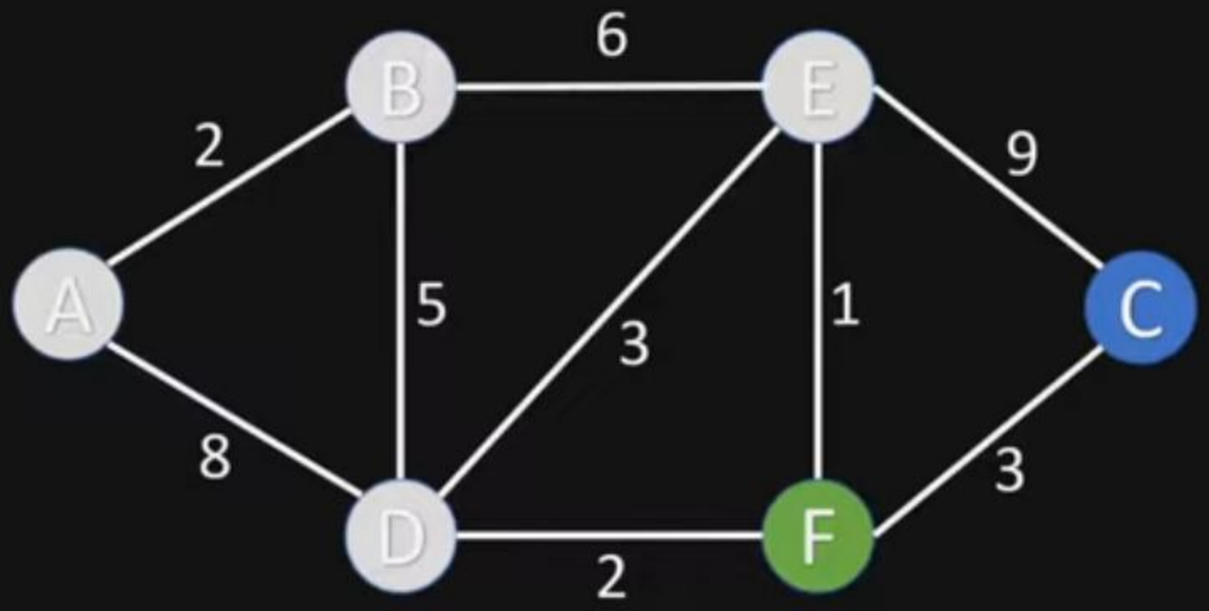
4. Mark current node as visited



Node	Shortest Distance	Previous Node
A	0	
B	**2**	A
C	**17**	E
D	**7**	B
E	**8**	B
F	**9**	D

Visited Nodes: [A, B, D, E] Unvisited Nodes: [C, F]

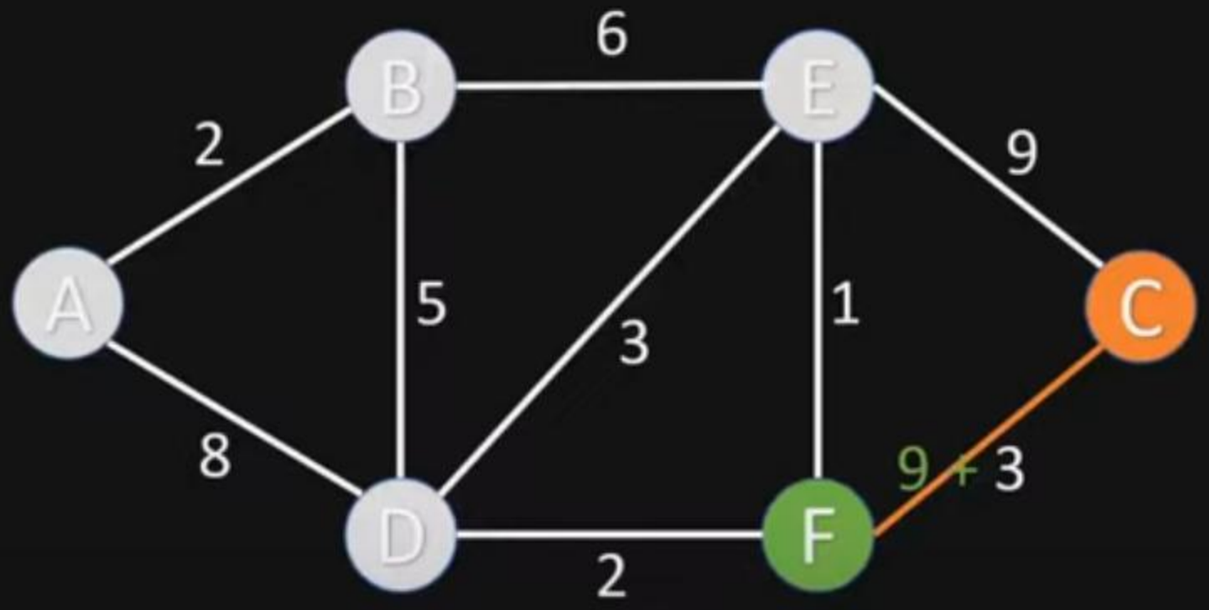
5. Choose new current node from unvisited nodes with minimal distance



Node	Shortest Distance	Previous Node
A	0	
B	**2**	A
C	**17**	E
D	**7**	B
E	**8**	B
F	**9**	D

Visited Nodes: [A, B, D, E] Unvisited Nodes: [C, F]

3. For the current node calculate the distance to all unvisited neighbours

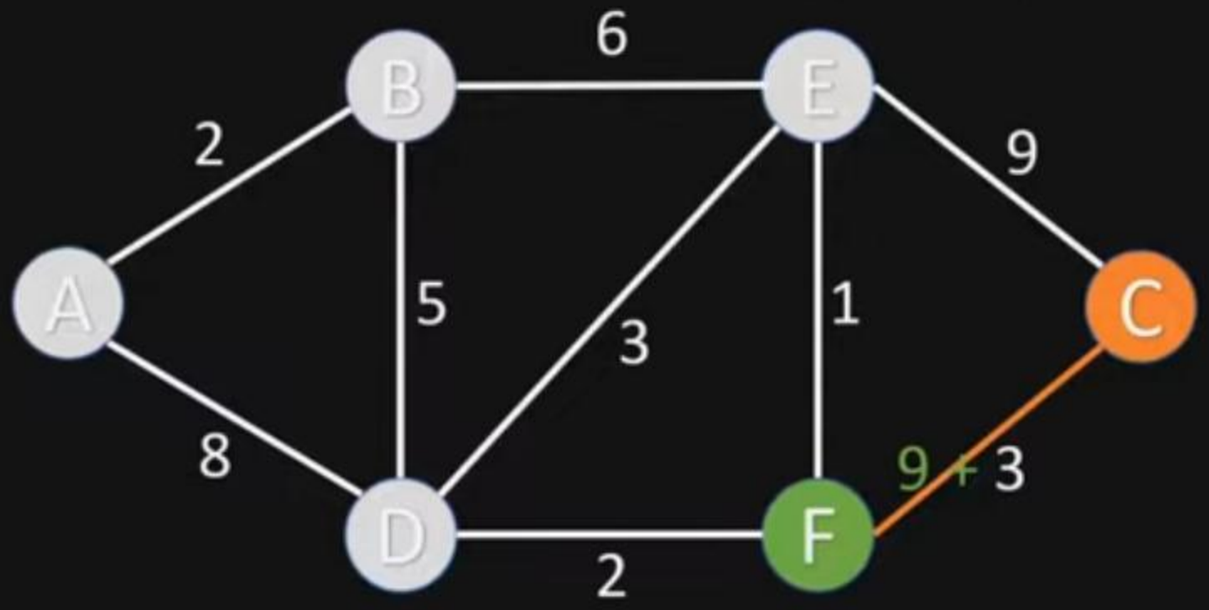


Node	Shortest Distance	Previous Node
A	0	
B	**2**	A
C	**17**	E
D	**7**	B
E	**8**	B
F	**9**	D

Visited Nodes: [A, B, D, E] Unvisited Nodes: [C, F]

3. For the current node calculate the distance to all unvisited neighbours

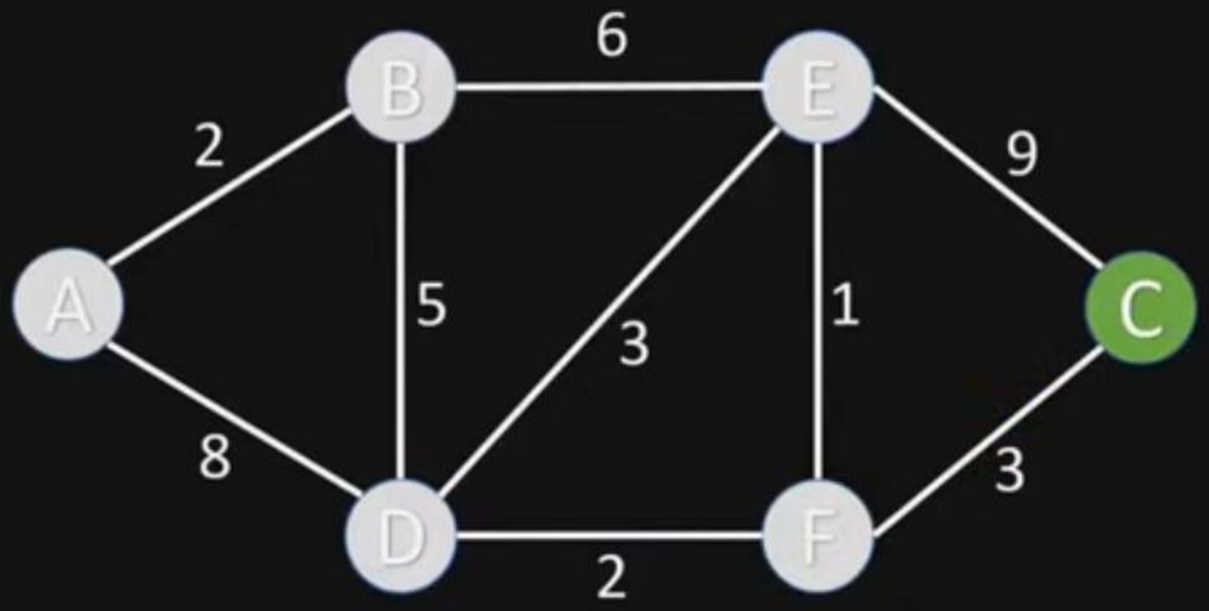
3.1. Update shortest distance, if new distance is shorter than old distance



Node	Shortest Distance	Previous Node
A	0	
B	**2**	A
C	**12**	F
D	**7**	B
E	**8**	B
F	**9**	D

Visited Nodes: [A, B, D, E] Unvisited Nodes: [C, F]

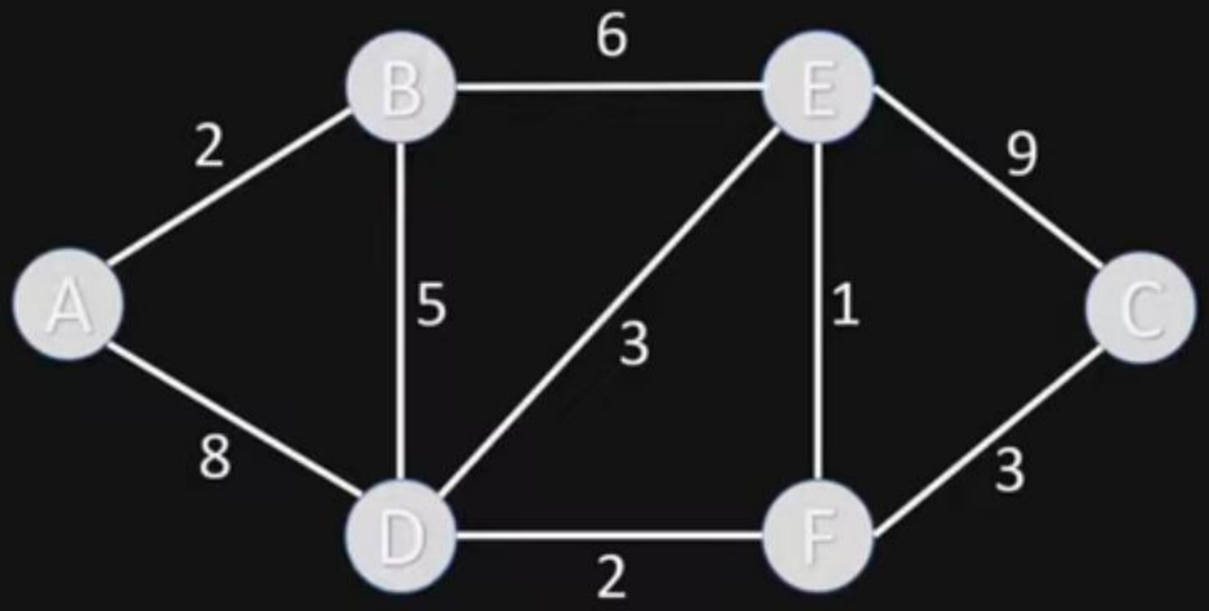
5. Choose new current node from unvisited nodes with minimal distance



Node	Shortest Distance	Previous Node
A	0	
B	**2**	A
C	**12**	F
D	**7**	B
E	**8**	B
F	**9**	D

Visited Nodes: [A, B, D, E, F] Unvisited Nodes: [C]

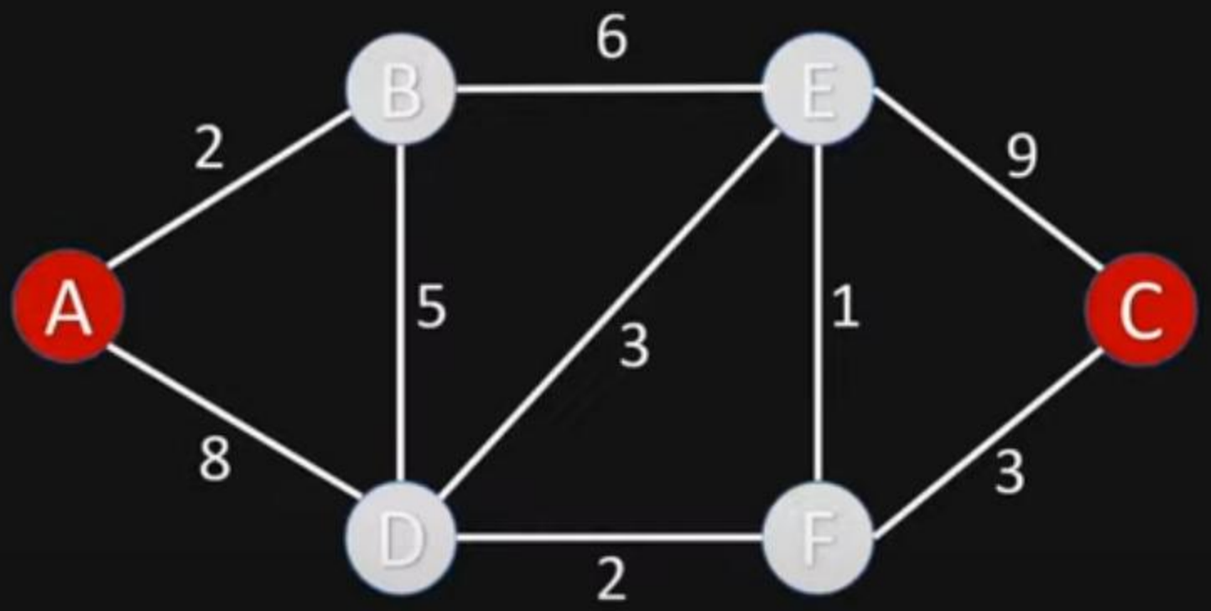
4. Mark current node as visited



Node	Shortest Distance	Previous Node
A	0	
B	**2**	A
C	**12**	F
D	**7**	B
E	**8**	B
F	**9**	D

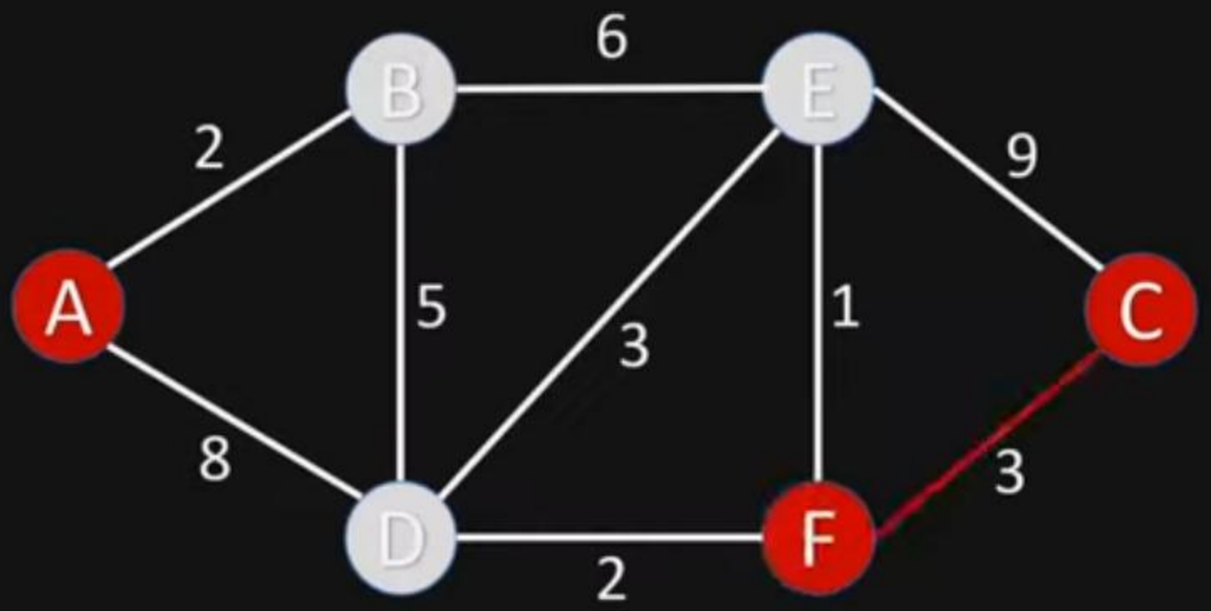
Visited Nodes: [A, B, D, E, F, C] Unvisited Nodes: []

Get shortest path from A to C



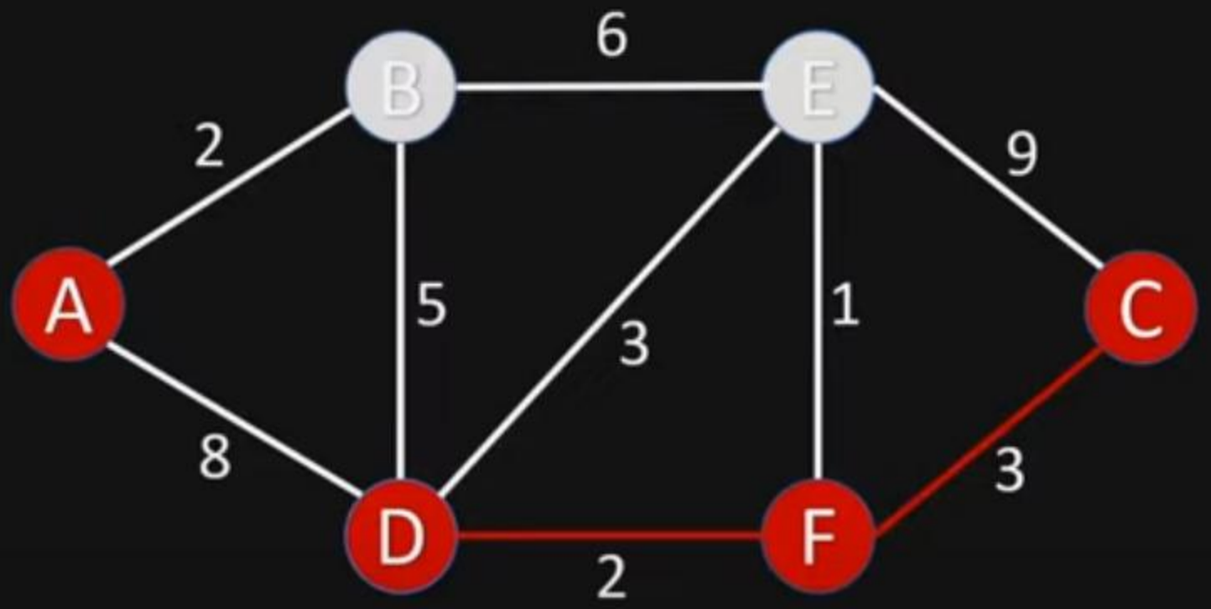
Node	Shortest Distance	Previous Node
A	0	
B	**2**	A
C	**12**	F
D	**7**	B
E	**8**	B
F	**9**	D

Get shortest path from A to C



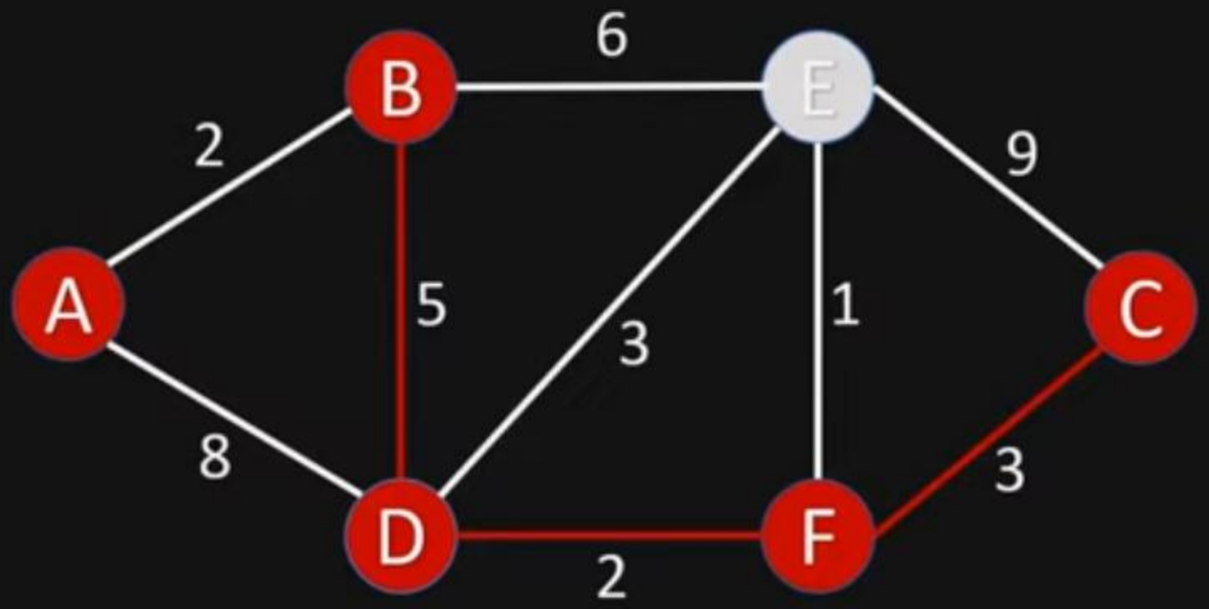
Node	Shortest Distance	Previous Node
A	0	
B	**2**	A
C	**12**	F
D	**7**	B
E	**8**	B
F	**9**	D

Get shortest path from A to C



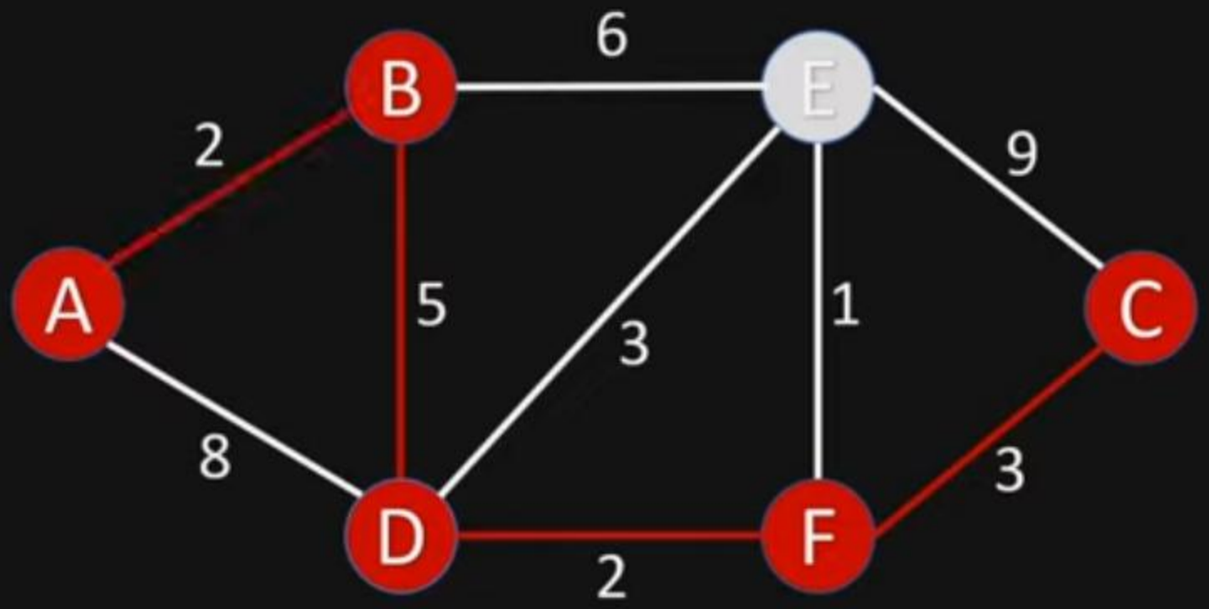
Node	Shortest Distance	Previous Node
A	0	
B	**2**	A
C	**12**	F
D	**7**	B
E	**8**	B
F	**9**	D

Get shortest path from A to C



Node	Shortest Distance	Previous Node
A	0	
B	**2**	A
C	**12**	F
D	**7**	B
E	**8**	B
F	**9**	D

Get shortest path from A to C



Node	Shortest Distance	Previous Node
A	0	
B	**2**	A
C	**12**	F
D	**7**	B
E	**8**	B
F	**9**	D